ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC VÀ KỸ THUẬT THÔNG TIN



GOOGLE SKILLS SCRAPER

Trích Xuất Nội Dung Của Google Skills Vào Kho Kiến Thức Cá Nhân (PKB)

Mã Môn:	IE221
Tên Môn:	Kỹ Thuật Lập Trình Python
Lớp:	IE221.F33.LT.CNTT
Giảng Viên:	ThS. Nghi Hoàng Khoa
MSSV:	25410291
Sinh Viên:	Đinh Xuân Sâm

Tháng 09/2026

TÓM TẮT BÁO CÁO

Đây là Báo Cáo Đồ Án của:

• Môn Học	**IE221 – Kỹ Thuật Lập Trình Python**
• Lớp	**IE221.F33.LT.CNTT**
• Giảng Viên	**Thạc Sĩ Nghi Hoàng Khoa**

Sinh viên thực hiện:

• Họ Tên	Đinh Xuân Sâm
• Mã Sinh Viên	25410291

Tiêu Chí

Báo Cáo này được thực hiện với các tiêu chí:

- Trình bày quá trình xây dựng một công cụ thực tế sử dụng Python.
- Trình bày các khối kiến thức đã được sử dụng từ môn học IE221 trong mã nguồn.
- Cung cấp mã nguồn của công cụ như một minh chứng của quá trình.
- Cung cấp bản demo như một minh chứng cho chức năng và khả năng hoạt động.
- Cởi mở và tường minh về các nội dung đã đạt được, chưa đạt được, hoặc không triển khai.
- Trình bày chuyên nghiệp theo hướng học thuật.

Nộp Bài

- Báo cáo: File PDF, khoảng 50 trang.
  - Vui lòng theo dõi Chương 3 (trang 12, gồm 20 trang) cho các nội dung về kiến thức trong môn học.
  - Các chương khác mang tính chất bổ sung hoặc/và làm rõ hơn các nội dung ở trong chương này.
- Slide: File PDF.
  - Như một bản rút gọn của báo cáo.
  - Hoàn toàn có thể bỏ qua báo cáo và vẫn trình bày đầy đủ nội dung cốt lõi.
- Demo: File hình ảnh/video đính kèm.
  - Phần DEMO được tách thành Phụ Lục để tránh loãng nội dung chính và sẽ được trình bày ở phụ lục Phụ Lục C (trang 47).
- Mã Nguồn: github.com/ggcta/skills-scrapper, nhánh `refactor-ie221`.
  - Được khởi tạo từ tháng 01/2025.
  - Và vẫn được duy trì hiện tại.
  - Hoàn toàn mở.
- Git Commit: File git log từ `git log --oneline` đính kèm.

Khai Báo Sử Dụng AI

- Bóc tách/chỉ ra các đoạn code đang có phản ánh các khối kiến thức của môn học để đưa vào báo cáo (xem ghi chú sau đây).
- Rà soát bộ khung nội dung báo cáo: chính tả, ngữ nghĩa (semantic).
- Thực hiện các commit chuẩn hóa ở nhánh `refactor-ie221` phục vụ cho báo cáo môn học, không thay đổi lõi logic của công cụ vốn đã ổn định.

Ghi Chú

- Sinh viên sử dụng `raw` của Typst để truyền đạt nguyên bản mã nguồn.
- Typst cho phép tải file (`load()`) trực tiếp để đảm bảo luôn có một nguồn sự thật duy nhất: có nghĩa nguồn trong code thế nào, sẽ được chuyển vào báo cáo như vậy.
- Đây là chức năng nguyên bản của trình soạn thảo/Typesetting Typst.

```
#raw(load("code/model/course.py"), lang: "python", block: true)
```

Tuyên Bố Miễn Trừ Trách Nhiệm

TUYÊN BỐ MIỄN TRỪ TRÁCH NHIỆM

Tất cả các nội dung của Google Skills đều là sở hữu của Google hoặc/và Google Skills. Không sao chép một phần hoặc tất cả nội dung cho mục đích khác ngoài mục đích ban đầu là phục vụ đào tạo các sản phẩm/dịch vụ của Google.

Công cụ này chỉ cung cấp một cách làm việc với các nội dung trên cho mục tiêu học tập cá nhân, KHÔNG CHỊU TRÁCH NHIỆM cho bất kỳ mục đích nào khác, và mọi quyền hạn hoặc trách nhiệm liên quan đến các nội dung được tải, vui lòng tham khảo Quy Định Sử Dụng của và trực tiếp tại Google Skills.

Đồng thời, Báo Cáo này không sao chép hoặc trình bày các nội dung của các khóa học/bài học, vv.. trên Google Skills mà chỉ tập trung vào khía cạnh kỹ thuật hoặc/và cấu trúc dữ liệu đằng sau.

Lời Nói Đầu

Google Skills (skills.google), trước đây là Google Cloud Skills Boost, là nền tảng đào tạo e-Learning chính thức của Google, bởi Google và cho các sản phẩm/dịch vụ của Google.

Vốn là nền tảng tập trung cho Google Cloud, nhưng bây giờ Google Skills đã mở rộng đối tượng sản phẩm phục vụ, và từ đó mở rộng số lượng nội dung được đăng tải, lên tới hàng trăm lộ trình học tập (path), hàng nghìn khoá học (course) và bài thực hành (lab).

Cách tổ chức của Google Skills cơ bản cũng tương tự các nền tảng e-learning khác, bao gồm: Lộ Trình Học Tập (Khóa Học) → Các Môn Học → Các Bài Học (Video, Bài Đọc, Bài Lab, vv..) và Các Bài Kiểm Tra (Quiz), Chứng Chỉ (Badge), vv.. rất bài bản và nhiều cấp bậc.

Phát sinh từ thực tế từ công việc cần làm việc trực tiếp với các dịch vụ của Google, cụ thể là Google Cloud, sinh viên thực hiện cũng cần sử dụng Google Skills để nhận các kiến thức liên quan về sản phẩm và dịch vụ, là một phần trong quá trình đào tạo (enablement) của công ty.

Tuy nhiên, sinh viên nhận thấy cách trình bày nội dung của Google Skills rất khó để theo dõi ở một số khía cạnh, ví dụ: Một bài học (course) hoặc bài lab có thể được sử dụng trong rất nhiều khóa học, tuy nhiên Google Skills không biểu diễn trực quan được mối quan hệ đó, khiến việc hình dung mối liên hệ kiến thức giữa các khóa học là rất khó, hoặc không thể, chỉ có thể tham gia hoặc đi qua từng khóa học để nhận ra là có môn học được dùng lại.

Đồng thời, việc bắt buộc sử dụng website để duyệt nội dung bài học, vốn chủ yếu là video youtube, không phải dạng văn bản, từ đó không hoặc khó cho phép việc ghi chú, tóm tắt, đánh dấu, vv.. về cơ bản là một cách tiếp cận không quá hiệu quả hay rất khó tham khảo lại.

Vì vậy, sinh viên đã đi đến quyết định thiết kế một công cụ nhỏ, mục đích kéo tất cả nội dung cần thiết của Google Skills về máy nội bộ, đóng gói thành các file thuần văn bản Markdown, trong một kho (vault) của Obsidian; vừa xây dựng cho mình một Kho Kiến Thức Cá Nhân (Personal Knowledge Base); vừa tạo tiền đề cho các sự mở rộng/liên kết kiến thức thậm chí bên ngoài Google Skills; hoặc/và đóng gói thành các văn bản dạng sách A4/A5, tiện lợi cho việc tra khảo hoặc đọc trên các thiết bị cầm tay (iPad/Kindle) cá nhân.

```
* commit 1c822f5eb80d5e6350f1e170379b8cabcb3af657
| Author: samdx <18287802+samdx@users.noreply.github.com>
| Date:   Sun Jan 5 17:07:57 2025 +0800
|
|     Init the repo.
|
* commit 9eb4e75a4ffa4c01cee6c36eadc2f8ce282747fe
  Author: samdx <18287802+samdx@users.noreply.github.com>
  Date:   Sun Jan 5 17:06:54 2025 +0800

      Create .gitattributes
```

Mục tiêu đầu tiên của công cụ ban đầu hoàn toàn không có ý tưởng về việc xây dựng một Bộ Não Thứ 2 (https://fortelabs.com/blog/basboverview/), tuy nhiên có một sự vô tình trong cách triển khai, mà sau này khi OKF (Open Knowledge Format) xuất hiện, mô hình nội dung trong công cụ đã tương thích gần như hoàn toàn.

Tuy nhiên, báo cáo này chỉ tập trung vào phần kỹ thuật của công cụ, các nội dung khác liên quan về cách tổ chức kho PKB sẽ không được đề cập ở đây hoặc chỉ được nhắc qua, ví dụ: backlink hoặc graph view trong Obsidian giúp trực quan hóa mối quan hệ giữa các khóa học/môn học; cách tổ chức kho theo mô hình PARA và cách khai thác, vv..

Hi vọng với các nội dung dưới đây, sẽ đem đến một vài tham khảo có ý nghĩa cho các bạn sinh viên cùng lớp, cũng như biểu hiện được các nội dung kiến thức đã được Giảng Viên truyền đạt trong môn học IE221 tại UIT.

Lời Cảm Ơn

Đầu tiên, sinh viên thực hiện xin cảm ơn sự hướng dẫn của Giảng Viên bộ môn IE221 – *Thạc Sĩ Nghi Hoàng Khoa*, đã nhiệt tình hướng dẫn, hỗ trợ tài liệu, công cụ và các bước thực hiện trong quá trình thực hiện Đồ Án này.

Tiếp theo, sinh viên thực hiện xin gửi lời cảm ơn tới trung tâm CITD – UIT, đã tạo môi trường cho các bạn sinh viên được tham gia học tập và sinh hoạt trong không khí của UIT.

Cuối cùng, xin gửi lời cảm ơn tới Google Skills, hay chính xác hơn trước đó là Google Skills Boost đã có một môi trường mở với lượng kiến thức chất lượng để sinh viên có thể tham khảo (mặc dù bây giờ đã đóng hơn rất nhiều) và từ đó phát sinh ý tưởng cho công cụ này.

Nhận Xét Của Giảng Viên

Ngày/Tháng

Giảng Viên

Mục Lục

TÓM TẮT BÁO CÁO i
Tiêu Chí i
Nộp Bài i
Khai Báo Sử Dụng AI ii
Tuyên Bố Miễn Trừ Trách Nhiệm ii
Lời Nói Đầu iii
Lời Cảm Ơn v
Nhận Xét Của Giảng Viên vi
Mục Lục vii
Danh Sách Bảng ix
Danh Sách Hình Ảnh x
Danh Sách Mã Nguồn xi
Bảng Viết Tắt xiii
1. MÔ TẢ BÀI TOÁN 1
1.1. Bối Cảnh 1
1.2. Vấn Đề Cần Giải Quyết 2
1.3. Mục Tiêu Đề Tài 3
1.4. Phạm Vi Ứng Dụng 3
1.5. Yêu Cầu 4
1.6. Kết Chương 5
2. TRIẾT LÝ VÀ THIẾT KẾ 6
2.1. Một Ứng Dụng, Không Phải Một Hệ Thống 6
2.2. Con Người Trong Vòng Lặp 7
2.3. Vault Như Bộ Não Thứ Hai 8
2.4. Đối Chiếu Với Open Knowledge Format 9
2.5. Kiến Trúc Gói 10
2.6. Hai Quyết Định về Thành Phần 11
2.7. Kết Chương 11
3. CÀI ĐẶT & TRIỂN KHAI 12
3.1. Môi Trường Phát Triển 12
3.2. Package Và Module 13
3.3. Mô Hình Hướng Đối Tượng 14
3.4. Kiểu Dữ Liệu, Hàm Và Cấu Trúc Điều Khiển 20
3.5. Xử Lý Ngoại Lệ 23
3.6. Phiên Đăng Nhập Và Thu Thập 25
3.7. Lưu Trữ 27

3.8. Kết Xuất Markdown 28
3.9. Hai Runner 30
3.10. Kết Chương 33
4. KIỂM THỬ, VẬN HÀNH VÀ BẢO TRÌ 34
4.1. Kiểm Thử Offline 34
4.2. Chạy Lại Và Tính Idempotent 35
4.3. Khi Nền Tảng Thay Đổi Và Hành Vi 36
4.4. Cấu Hình Và Tính Di Động 37
4.5. Kết Chương 38
5. KẾT LUẬN 39
5.1. Kết Quả Đạt Được 39
5.2. Hạn Chế 41
5.3. Hướng Phát Triển 41
5.4. Các Bài Học Từ Dự Án 42
5.5. Lời Kết 42
A Công Cụ Sử Dụng 43
A.1 Mã Nguồn 43
A.2 Báo Cáo 43
A.3 Khai Báo Sử Dụng AI 43
B Nguồn Gốc Và Tư Liệu 44
B.1 Mã Nguồn 44
B.2 Danh Sách Commit Refactor 44
B.3 Kịch Bản Demo 45
B.4 Chạy Từ Bản Nộp Bài 45
C DEMO 47
Tài Liệu Tham Khảo 51

Danh Sách Bảng

Bảng 1 Google Skills – Phân Cấp Dữ Liệu 1
Bảng 2 Một Số Câu Hỏi Thường Gặp 2
Bảng 3 Yêu Cầu Chức Năng 4
Bảng 4 Yêu Cầu Phi Chức Năng 4
Bảng 5 Hai Portal và Danh Mục Tương Ứng (09/2026) 8
Bảng 6 OKF v0.2 – Đối Chiếu Các Yêu Cầu 9
Bảng 7 `skills-scraper – cli` 30
Bảng 8 Kiểm Thử – Bộ Test Offline 34
Bảng 9 Khi Nền Tảng Thay Đổi Và Hành Vi 36
Bảng 10 Kết Quả Đạt Được 39
Bảng 11 Đối Chiếu Với Kiến Thức Môn Học 40
Bảng 12 Phụ Lục – Kịch Bản Demo 45

Danh Sách Hình Ảnh

Hình ảnh 1 Obsidian – Graph View Của Vault 9
Hình ảnh 2 Kiến Trúc Gói 10
Hình ảnh 3 `model/` – `Serialize` và Sơ Đồ Lớp 14
Hình ảnh 4 Hai Cây Kế Thừa Trong Mã Nguồn 16
Hình ảnh 5 `skills-scraper` Trong Terminal 31
Hình ảnh 6 `skills-scraper-tui` – Cùng Lõi `model` – Khác Giao Diện 32

Danh Sách Mã Nguồn

Mã nguồn 1 Google Skills – Cây Phân Cấp Dữ Liệu 2
Mã nguồn 2 Frontmatter của note course 892 ở v2.0.0 10
Mã nguồn 3 `pyproject.toml` – Tên, Phiên Bản, Các Thư Viện 12
Mã nguồn 4 `pyproject.toml` – Hai Runner và Build Toolkits 12
Mã nguồn 5 `.python-version`: Phiên Bản Python Tối Thiểu 13
Mã nguồn 6 `__init__.py`: Phiên Bản Đọc Từ Metadata Của Gói 13
Mã nguồn 7 Cây Package `src/skills_scraper/` 14
Mã nguồn 8 `model/serialize.py`: Gốc Của Cây Kế Thừa 15
Mã nguồn 9 `BaseEntity.__init__()` – Ba Thuộc Tính Định Danh 16
Mã nguồn 10 `BaseEntity.type` và `BaseEntity.url` 17
Mã nguồn 11 `BaseEntity.generate_markdown` – Mặc Định Chỉ Có Frontmatter 17
Mã nguồn 12 `BaseEntity.save_markdown`: gọi `generate_markdown` Của Lớp Con 18
Mã nguồn 13 `model/lab.py` – Kế Thừa `BaseEntity` 18
Mã nguồn 14 `model/lab.py` – Ghi Đè `generate_markdown` 18
Mã nguồn 15 `model/courses.py` – Lớp Con Chỉ Khai `API_URL` 19
Mã nguồn 16 `BaseEntity.to_dict()` – Lưu Gì Ra File JSON 19
Mã nguồn 17 `Course.process_lab()` – Tạo `Lab`, Nạp JSON 20
Mã nguồn 18 `Course.process_lab()` – Lưu `Lab` Vừa Tải 20
Mã nguồn 19 `Course.process_lab()` – Cập Nhật Tập Hợp `Labs` 20
Mã nguồn 20 `Course.__init__()` – Kiểu Dữ Liệu 21
Mã nguồn 21 `Course.process_step()` – Bảng `handlers` 21
Mã nguồn 22 `Course.process_step()` – Loại Lạ Được Giữ Nguyên và Thông Báo 22
Mã nguồn 23 `Course._parse_lesson_item()`: Lặp Đệ Quy Trên Cây *item* . . 22
Mã nguồn 24 `Collection.fetch_catalog()` – Bắt Đầu 22
Mã nguồn 25 `Collection.fetch_catalog()` – Vòng Lặp Có Điểm Ngắt 23
Mã nguồn 26 `store.write_text()` – Ghi Nguyên Tử Với `try/except/raise` 24
Mã nguồn 27 `browser.ensure_authenticated()` – `KeyboardInterrupt`, `EOFError` 24
Mã nguồn 28 `BaseEntity.save_json()` – Ghi File Trước, Cập Nhật Chỉ Mục Sau 25
Mã nguồn 29 `browser.launch_browser()` – Khởi Chạy Trình Duyệt 26
Mã nguồn 30 `browser.open_page()`, `browser.get_page()` – Duyệt Trang . . 26

Mã nguồn 31 `Course.extract_course_metadata()` – Bỏ Qua Theo `datePublished` 27
Mã nguồn 32 `data/` – Mỗi Thực Thể Một File 27
Mã nguồn 33 `store.write_text()` – Lưu File 28
Mã nguồn 34 `892.md` – Frontmatter Từ JSON 29
Mã nguồn 35 `BaseEntity.generate_front_matter()` – Tạo Frontmatter 29
Mã nguồn 36 CLI – Kết Hợp Với `jq`, `while` của Shell và `git` 30
Mã nguồn 37 `CloudSkillsBoost.driver()` – Khởi Tạo Trình Duyệt 32
Mã nguồn 38 `tui.main` – `try/finally` – Đóng Trình Duyệt Khi Thoát 32
Mã nguồn 39 `uv sync` và `uv run pytest` 35
Mã nguồn 40 `config.load_settings()` – Tìm và Tải `config.yaml` 37
Mã nguồn 41 `git log --oneline v2.0.0..refactor-ie221` – Ví Dụ Các Thay Đổi 45
Mã nguồn 42 Phụ Lục – Các Lệnh Để Chạy Từ Bản Nộp Kèm 46

Bảng Viết Tắt

Viết Tắt	Nghĩa Đầy Đủ
ANSI	American National Standards Institute (mã màu terminal)
API	Application Programming Interface
CLI	Command Line Interface (giao diện dòng lệnh)
CODE	Capture, Organize, Distill, Express (khung Bộ Não Thứ Hai)
CSB	Cloud Skills Boost (tên cũ của Google Skills)
CSV	Comma-Separated Values
CTA	Cloud Technical Architect
DOM	Document Object Model
FR	Functional Requirement
GCP	Google Cloud Platform
GUI	Graphical User Interface
HTML	HyperText Markup Language
JSON	JavaScript Object Notation
KISS	Keep It Simple, Stupid
LLM	Large Language Model
NFR	Non-Functional Requirement
OKF	Open Knowledge Format (Google)
OOP	Object-Oriented Programming
PARA	Projects, Areas, Resources, Archives
PDF	Portable Document Format
PEP	Python Enhancement Proposal
PKB	Personal Knowledge Base
SSOT	Single Source Of Truth
TOC	Table Of Contents
TOML	Tom's Obvious, Minimal Language
TUI	Text User Interface
URL	Uniform Resource Locator
UTF-8	Unicode Transformation Format 8-bit
YAML	YAML Ain't Markup Language

CHƯƠNG 1.

MÔ TẢ BÀI TOÁN

Trong chương này, sinh viên thực hiện sẽ trình bày ngữ cảnh, bài toán cần giải quyết, và các câu hỏi đặt vấn đề cho một công cụ. Từ đó, khoanh vùng phạm vi các mục tiêu cần thực hiện cũng như các nội dung được và không được đề cập trong báo cáo này.

1.1. Bối Cảnh

1.1.1. Giới Thiệu Google Skills

Như đã đề cập, Google Skills (skills.google), trước đây là Google Cloud Skills Boost, là nền tảng đào tạo chính thức của Google Cloud, gồm hàng trăm lộ trình học (path), hàng nghìn khoá học (course) và bài thực hành (lab).

Trong khoảng hai năm, từ cuối 2024 đến đầu 2026, nền tảng đã có những thay đổi trong cả định hướng hoạt động và trong bộ nội dung được triển khai, và từ đó ảnh hưởng trực tiếp tới cách thu thập nội dung, ví dụ: từ một nền tảng mở, Google Skills bây giờ đã yêu cầu phải có tài khoản đăng nhập để có thể xem nội dung môn học, từ đó một hệ quả kỹ thuật phát sinh: thư viện `requests` không còn đủ, việc thu thập phải đi qua một phiên trình duyệt đã đăng nhập (Selenium), vì vậy công cụ đã thay đổi hành vi đáng kể so với ngày đầu tiên được tạo dựng.

1.1.2. Phân Cấp Dữ Liệu

Để phục vụ cho đăng tải nội dung tới nhiều tệp người dùng, cũng như nhiều tệp sản phẩm, phân cấp dữ liệu trên Google Skills có thể tóm tắt như bảng dưới đây.

Cấp	Thực thể	Ghi chú
1	**Portal**	Bao gồm *public* và *partner*; báo cáo này chỉ đề cập *public* portal vì *partner* là dành cho đối tác của Google, và cần có tài khoản riêng.
2	**Path**	Lộ Trình, chứa nhiều Course và Lab theo thứ tự.
3	**Course**	Khoá Học, có mô tả, mục tiêu, chủ đề và từng chương.
4	**Module**	Chương, thuộc Khoá Học, sẽ có từng bài học cụ thể.
5	**Step**	Các Bước/Phần trong chương.
6	**Activity**	Các Bài Học, có thể là video, lab, quiz, pdf hoặc html, hoặc loại nội dung bất kỳ nào khác.

Bảng 1: Google Skills – Phân Cấp Dữ Liệu

Mô hình phân cấp có thể được trực quan hóa đơn giản như sau:

```
Portal (public | partner)
└─ Path (learning path)
  └─ Course
    └─ Module
      └─ Step
        └─ Activity
          └─ video | lab        | quiz        |
          └─ link  | document   | html_bundle |
          └─ badge | credential | ...
```

Mã nguồn 1: Google Skills – Cây Phân Cấp Dữ Liệu

> **Ghi Chú**
>
> - Lab vừa là một hoạt động trong Course vừa là một thực thể độc lập, đôi khi được tham chiếu trực tiếp bởi Path, điều này đôi lúc tạo nên sự phức tạp trong logic của việc liên kết các thực thể.
> - Tập các loại hoạt động (**Activity**) thuộc về Google chứ không thuộc về ứng dụng: Google Skills có thể thay đổi các loại hoạt động bất kỳ lúc nào, hiện tại thường là sẽ thêm mới dựa theo cách triển khai nội dung của các khóa học/môn học mới.

1.2. Vấn Đề Cần Giải Quyết

Website Google Skills cho người dùng xem một trang tại một thời điểm. Một số câu hỏi người học hay đặt ra nhất lại không trả lời được trên bản thân trang độc lập đó. Để so sánh, đây là các tính năng có sẵn của Obsidian khi nội dung là Markdown liên kết:

Câu hỏi	skills.google	Obsidian Vault
Course này **thuộc những Path** nào?	phải tìm ngược từng path	backlink/wikilink
Hai Path **chồng lấn** (dùng chung) **bao nhiêu** Course?	không có	graph view, truy vấn
Transcript/Course nào **nhắc tới một khái niệm** cụ thể?	khó tìm trong transcript	tìm toàn văn, offline
Course này đã **thay đổi gì** từ lần học trước?	không có lịch sử	`date_published` và `git diff`

Bảng 2: Một Số Câu Hỏi Thường Gặp

Quy trình hiện tại của người học trên Google Skills: duyệt danh mục, mở từng khoá, xem video, ghi chú tay. Nội dung nằm rải rác qua video, quiz, lab, tài liệu; không có bản xuất (export) để lưu lại, không có bản offline, không tìm kiếm xuyên khoá. Có một sự rời rạc trong một hệ thống được thiết kế trông rất chặt chẽ.

1.3. Mục Tiêu Đề Tài

Mục tiêu đầu tiên, công cụ thực tiễn:

- Xây dựng một ứng dụng/công cụ Python dòng lệnh.
- Thu thập Path, Course, Lab từ skills.google vào kho JSON (file-based database).
- Sinh ra một vault Markdown dùng được ngay trong Obsidian
- Cung cấp hai giao diện terminal (CLI cho tự động hoá, TUI cho tương tác).
- Hoàn toàn có thể mở rộng sang các nền tảng khác.

Mục tiêu thứ hai, về mặt học thuật:

- Là dùng chính ứng dụng này làm chứng cứ cho các kiến thức của môn học IE221: **module** và **package**, **lớp** và **đối tượng**, **kế thừa**, xử lý ngoại lệ, vv... trong Python.
- Như một tiền đề để phát triển các chức năng hay ứng dụng như *một phần của hệ thống* vốn thường gặp trong thực tế khi các công cụ thường là một phần của một hệ sinh thái lớn với nhiều công cụ riêng lẻ.

1.4. Phạm Vi Ứng Dụng

Các phạm vi chức năng sẽ được/không được triển khai, tương ứng như sau đây.

1.4.1. Bao Gồm

Đây là các chức năng được triển khai trực tiếp, và đã sẵn sàng.

Thành Phần	Nội Dung
• Portal	Public
• Entity	Path, Course, Lab
• Activity	video transcript, quiz, lab, link, pdf, html_bundle
• Output	JSON, Markdown
• Runner	CLI, TUI

1.4.2. Không Bao Gồm

Đây là các chức năng không được triển khai, hoặc đã loại bỏ, hoặc chưa từng được đề cập trong quá trình xây dựng.

Thành Phần	Nội Dung
• Portal	Partner (sinh viên không còn quyền truy cập)
• Automation	Chạy (hoàn thành) Lab (phần thực hành của người học)
• Account	Quản lý tài khoản, mua khoá học
• GUI	Không (lý do sẽ được trình bày ở Chương 2 (trang 11)

Thành Phần	Nội Dung
• Database	Không (lý do sẽ được trình bày ở Chương 2 (trang 11)

1.5. Yêu Cầu

1.5.1. Yêu Cầu Chức Năng

Mã	Yêu cầu
FR-01	Liệt kê danh mục `path`, `course`, `lab`; làm mới từ site hoặc đọc offline
FR-02	Thu thập một `path` và lan xuống các `course`, `lab` của nó
FR-03	Thu thập một `course` với đầy đủ `module`, `step`, `activity`
FR-04	Thu thập một `lab` độc lập
FR-05	Bỏ qua course có `datePublished` không đổi, trừ khi `--force`
FR-06	Sinh Markdown cho `path`, `course`, `lab`; tuỳ chọn `--toc`, `--no-transcript`
FR-07	Tìm kiếm theo tên (`name`), `id`, chủ đề (`topic`)
FR-08	Dựng lại chỉ mục từ dữ liệu JSON (`reindex`)
FR-09	Tải tài liệu PDF vào `materials/courses/<id>/`
FR-10	Sinh tệp prompt JSON cho mô hình ngôn ngữ lớn (LLM).
FR-11	Giao diện tương tác dạng menu (TUI)

Bảng 3: Yêu Cầu Chức Năng

> **Note**
>
> FR-10: Sinh tệp prompt JSON cho mô hình ngôn ngữ lớn (LLM) để tóm tắt nội dung transcript, đã từng được đề xuất, nhưng giờ tạm ngừng xây dựng.

1.5.2. Yêu Cầu Phi Chức Năng

Mã	Yêu cầu
NFR-01	Không trạng thái (stateless): mỗi lần chạy đọc tệp, ghi tệp; chạy lại là *idempotent* (nguyên bản)
NFR-02	Dữ liệu và đầu ra là văn bản thuần (plain text), UTF-8, đọc được bằng `git diff`
NFR-03	Cấu hình tường minh qua biến môi trường và tệp văn bản (`config.yaml`)
NFR-04	Lỗi tường minh: một `activity` hỏng không làm hỏng cả `course`, mọi ngoại lệ đều được báo
NFR-05	Ghi tệp nguyên tử (`atomic`): nếu gặp ngắt giữa chừng không để lại nửa tệp lỗi
NFR-06	Môi trường tái lập: `uv sync` từ một bản clone sạch là có thể bắt đầu

Bảng 4: Yêu Cầu Phi Chức Năng

1.6. Kết Chương

Chương 1 này đã mô tả một bài toán thực tế của sinh viên: nội dung học tập nằm trên một nền tảng chỉ cho xem từng trang, ngày càng đóng, và không trả lời được những câu hỏi về liên kết giữa các khoá học. Nhu cầu phát sinh từ công việc, nên lời giải là một công cụ thực tế với các một yêu cầu chức năng (Bảng 3) và các yêu cầu phi chức năng (Bảng 4) tương ứng, có những đặc thù nhất định và không nhất thiết dùng hết mọi kiến thức của môn học IE221. Chương 2 sẽ trình bày triết lý và các quyết định thiết kế để trả lời bài toán đó bằng một ứng dụng nhỏ.

CHƯƠNG 2.

TRIẾT LÝ VÀ THIẾT KẾ

Trước khi đi vào mã nguồn, trong chương này, sinh viên thực hiện xin trình bày các điểm về tư duy xây dựng và các quyết định về thiết kế đã ảnh hưởng đến sự hành thành của công cụ: vì sao nó là một ứng dụng nhỏ chứ không phải một hệ thống, vì sao con người vẫn ở trong vòng lặp khi site thay đổi, vault sinh ra đóng vai trò gì trong mô hình Bộ Não Thứ Hai và khớp với Open Knowledge Format của Google ở đâu, kiến trúc gói được chia thế nào, và cuối cùng là hai quyết định cố ý không dùng (cơ sở dữ liệu, giao diện đồ hoạ) dù đó là hai chương trong khung kiến thức của môn học.

2.1. Một Ứng Dụng, Không Phải Một Hệ Thống

Công cụ được thiết kế dưới góc nhìn của kỹ sư hệ thống và theo triết lý Unix *[1]*: một thành phần nhỏ, làm đúng một việc, ghép được với các công cụ khác; mà không phải là một "hệ thống" hay một nền tảng hoàn toàn độc lập. Năm nguyên tắc dưới đây quyết định mọi lựa chọn trong các chương sau.

2.1.1. Làm Một Việc Và Làm Tốt

> *KISS*: **Keep It Simple, Stupid**
>
> - Làm một việc, và làm thật tốt việc đó.
> - Hoàn toàn có thể kết hợp với các công cụ khác để tạo thành một luồng công việc.
> - Dễ dàng được thay thế mà không ảnh hưởng đến tổng thể.

Ứng dụng chỉ thu thập và chuẩn hoá.

- Lệnh `fetch` đưa nội dung về JSON.
- Lệnh `md` chuyển JSON thành Markdown, hai lệnh không (cần) biết nhau (độc lập nội tại).
- Ba thư viện ngoài lúc chạy (selenium, beautifulsoup4, requests).
- Không daemon, không server.

2.1.2. Đơn Giản Để Luôn Thay Thế Được

Thứ bền nhất không phải công cụ mà là dữ liệu nó để lại. JSON và Markdown thuần vẫn đọc được khi công cụ không còn tồn tại.

Dự án đã tự chứng minh: dòng chính của repo thay lõi Python bằng lõi Go (thử nghiệm) mà vault không đổi định dạng. Một công cụ khoá dữ liệu của người dùng vào chính nó thì không phải là khó thay thế, mà biến dữ liệu thành con tin. Đây cũng là một góc nhìn khác của "mã nguồn mở".

> **Tiêu Chí**
>
> - Đơn giản đến mức có thể được thay thế bất kỳ lúc nào.
> - Hiệu quả đến mức không cần phải thay thế.

2.1.3. Là Một Phần Của Hệ Thống

Ứng dụng giao tiếp bằng tệp văn bản thuần (plain text) và mã thoát (exit code), nên ghép được với `grep`, `jq`, `git`, `cron`, Obsidian và các plugin AI mà không cần API riêng. Đồ thị của khoảng 3000 note trong vault không được thiết kế trước: nó hình thành từ các liên kết tương đối mà mỗi note tự ghi ra.

> **Emergence**
>
> Nguyên Lý Đột Sinh (Emergence) – một phần "ngốc nghếch" của một hệ thống *thông minh*.

2.1.4. Tường Minh

- Cấu hình nằm trong biến môi trường và tệp `config.yaml` văn bản thuần, có tệp mẫu trong repo.
- Mỗi note Markdown tự khai nó được sinh bởi phiên bản nào, từ trang nào, trang đó sửa lần cuối khi nào đều được nêu rõ, ví dụ bằng YAML frontmatter.
- Khi bỏ qua một `course` vì course đó không đổi, ứng dụng nêu rõ ra là đã bỏ qua (course nào) và vì lý do gì.

2.1.5. Thêm Hay Không Thêm

Mỗi thư viện và mỗi thành phần thêm vào phải trả lời được câu hỏi "nó đóng góp được gì".

Quy ước của dự án: một thư viện chỉ được khai báo trong đúng *commit* đầu tiên `import` nó; cơ sở dữ liệu lớn chỉ xứng đáng khi cần hiệu năng cao, ghi đồng thời song song, hoặc xử lý phân tán; do bài toán này không cần cả ba, vì vậy đây là lý do báo cáo này không sử dụng một số kiến thức nhất định trong môn học.

2.2. Con Người Trong Vòng Lặp

- Câu hỏi: *nếu Google thay đổi cấu trúc website hay khoá học, ứng dụng có tự phát hiện và thay đổi theo không?*

- Câu trả lời tách làm hai vế: phát hiện thì **có**, tự thích nghi/thay đổi thì **không**, và đó là chủ ý thiết kế của người thực hiện.

Về phát hiện: loại *activity* lạ được giữ nguyên dữ liệu thô và ghi log:

- Thiếu metadata hoặc đề cương thì dừng course đó và báo rõ
- Phiên đăng nhập hết hạn thì dừng chờ người dùng.
- Bộ mẫu HTML trong `tests/` sẽ kích hoạt một thay đổi markup (của Google) thành một test thất bại (và từ đó cần xử lý tương ứng).

Về không tự thích nghi, có ba lý do.

- Một scraper tự sửa selector thực chất là **đoán**, và đoán sai sẽ ghi dữ liệu sai vào vault một cách âm thầm; *lỗi im lặng nguy hiểm hơn lỗi ồn ào*, thà để ồn ào để biết đó là gì, còn hơn để mọi thứ sai trong im lặng, đây cũng là chủ ý thiết kế.
- Có thực hiện thay đổi hay không là quyết định về giá trị: nếu mục tiêu cốt lõi (lấy nội dung về) vẫn đạt, thời gian, công sức và tài nguyên tính toán bỏ ra có xứng đáng không; bản thân logic của công cụ không trả lời được câu đó, và đó là lúc con người tham gia trong "vòng lặp".

Mỗi thay đổi của site là một commit có nguồn và có lý do. Mô hình triển khai: công cụ báo lỗi, người quyết định, git ghi lại lịch sử bao gồm cả lý do, cách xử lý, vv... thành một nguồn sự thật duy nhất (SSOT – Single Source Of Truth).

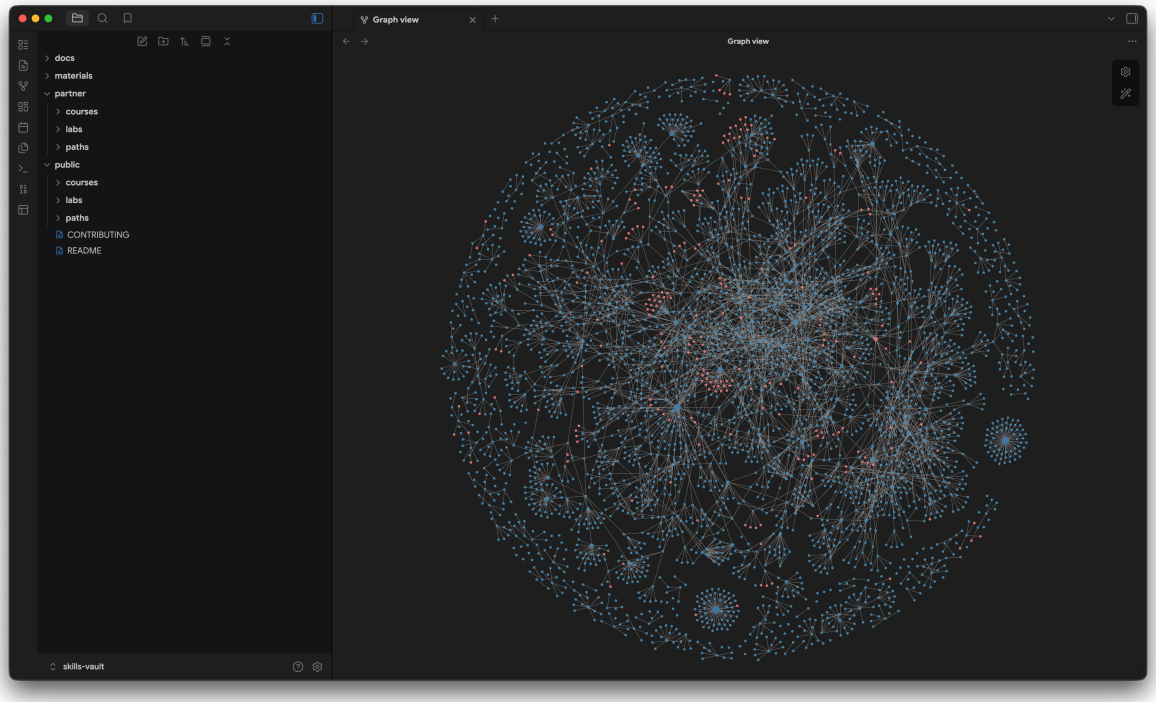
2.3. Vault Như Bộ Não Thứ Hai

Theo khung CODE (Capture, Organize, Distill, Express) của Tiago Forte *[2]*, ứng dụng làm **Capture** và một phần **Distill** (`--toc` , `--no-transcript` , tệp prompt). **Organize** thuộc về Obsidian, **Express** thuộc về người học. Ứng dụng cố ý và nên dừng ở mức đó.

Portal	Paths	Courses	Labs
public	73	416	824
partner	181	899	1209

Bảng 5: Hai Portal và Danh Mục Tương Ứng (09/2026)

Obsidian – Graph View Của Vault: mỗi nút là một thực thể, mỗi cạnh là một liên kết tương đối mà note tự ghi ra; càng nhiều liên kết (và thường kích thước càng lớn), có nghĩa thực thể đó càng được tham chiếu tới, có nghĩa có lượng ảnh hưởng càng lớn trên tổng nội dung của Google Skills.



Hình ảnh 1: Obsidian – Graph View Của Vault

2.4. Đối Chiếu Với Open Knowledge Format

OKF v0.2 *[3]* chỉ bắt buộc mọi tệp `.md` có frontmatter YAML đọc được với khoá `type` không được rỗng. Vault đã thoả điều này từ 2025 vì `type` được suy ra từ tên lớp. Việc trùng khớp không phải ngẫu nhiên: cả hai cùng xuất phát từ ràng buộc "một khái niệm là một tệp, siêu dữ liệu (metadata) ở đầu, thân bài cho người đọc".

> **Fun Fact**
>
> Thực ra cách trình bày file Markdown này không mới, là kế thừa từ thời sử dụng Jekyll.

OKF v0.2	Vault v2.0.0	Trạng thái
`type` (bắt buộc)	`type`	đã có
`title`	`title`	đã có
`resource`	`url`	khác tên, giữ làm alias
`tags`	`topics`	khác tên, giữ làm alias
`description`	mô tả trong thân bài	bổ sung
`generated {by, at}`	`scraped_date`	bổ sung
`sources[].last_modified`	`date_published`	bổ sung
`index.md`, `log.md`	không có	hướng phát triển

Bảng 6: OKF v0.2 – Đối Chiếu Các Yêu Cầu

Frontmatter của một thực thể ở v2.0.0, trước khi đối chiếu; bản sau khi mở rộng được trình bày ở Phần 3.8:

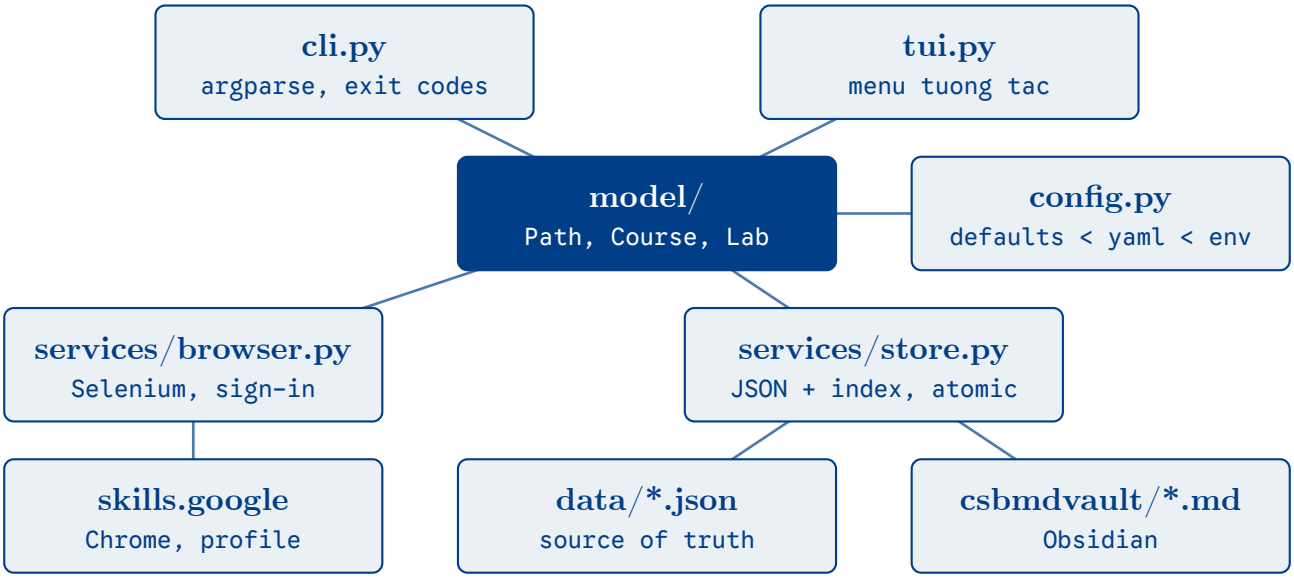
```
"id": "892"
name: "Conversational AI on Vertex AI and Dialogflow CX"
type: "Course"
url: "https://www.cloudskillsboost.google/course_templates/892"
description: "In this course you will learn how to use the new
generative AI features..."
date: "2025-01-06"
datePublished: "2023-11-21",
```

Mã nguồn 2: Frontmatter của note course 892 ở v2.0.0

2.5. Kiến Trúc Gói

```
src/skills_scraper/
  __init__.py
  config.py     cấu hình, URL, selector
  cli.py        runner dòng lệnh
  tui.py        runner tương tác
  model/        LÕI: Serialize, BaseEntity, Collection và các lớp con
  services/     browser.py (phiên đăng nhập), store.py (lưu trữ)
  utils/        xử lý văn bản, ghi tệp nguyên tử
```

Hướng phụ thuộc một chiều: runner gọi `model/`, `model/` dùng `services/` và `utils/`; `services` làm việc với file và trình duyệt; `utils` thực hiện một số tác vụ rất nhỏ.



Hình ảnh 2: Kiến Trúc Gói

2.6. Hai Quyết Định về Thành Phần

> **Ghi Chú**
>
> Đây đều là các quyết định thiết kế có lý do và chủ đích, mà không phải là sự vô tình bỏ sót kỹ năng đã học hoặc nội dung chương trình.

2.6.1. Không Cơ Sở Dữ Liệu (Chương 10 - IE221)

Cơ Sở Dữ Liệu không được sử dụng với các lý do sau đây, là về chức năng không phải vì không thể triển khai.

- Kho có dưới 5.000 bản ghi, Tìm Kiếm Tuyến Tính có thể trả về kết quả trong mili giây.
- Tệp phẳng (flat-file) và văn bản thuần (plain text), nên đọc được bằng git và Obsidian hoặc bất kỳ công cụ nào khác.
- Không trạng thái (stateless) nên chạy ở đâu cũng được.

> **Ghi Chú**
>
> Phiên bản v2.0.0 từng dùng song song JSON và TinyDB, và hiện đã bỏ TinyDB ở các phiên bản sau, vì không xứng đáng để bảo trì một lúc hai nguồn sự thật (flat-file trong git và database trong TinyDB) trong khi không đem lại giá trị quá khác biệt.

2.6.2. Không Giao Diện Đồ Hoạ (Chương 09 – IE221)

Tương tự, GUI không được triển khai vì đặc thù nhu cầu không sử dụng. Điều này không có nghĩa GUI không thể thực hiện, thậm chí với kiến trúc hiện tại, GUI có thể được thêm vào như một phần đóng góp nếu người khác có nhu cầu.

- Người dùng và môi trường mục tiêu là người làm việc trong terminal.
- Giao diện đồ hoạ thật đã tồn tại ở nhánh Tauri (như một ví dụ về khả năng mở rộng) nhưng sẽ không được ghép vào dòng/chức năng chính.
- Báo cáo này sẽ chỉ tập trung vào phần lõi của công cụ (model) nơi biểu hiện rõ nhất các nội dung của chương trình học.

2.7. Kết Chương

Chương này trình bày các lý do và quyết định trong quá trình xây dựng công cụ. Đây là tiền đề cho Chương 3, nơi các nội dung cốt lõi của môn học được chỉ ra trên chính mã nguồn đã chọn theo các nguyên tắc đã được trình bày ở đây.

CHƯƠNG 3.

CÀI ĐẶT & TRIỂN KHAI

Đây là phần nội dung chính của báo cáo, và để dễ theo dõi, cũng như bám sát các khối kiến thức của môn học IE221, sinh viên sẽ trình bày lần lượt các nội dung tương ứng.

3.1. Môi Trường Phát Triển

`uv` *[4]* được dùng để khai báo, quản lý gói và khoá phiên bản:

- Nhất quán, có thể tái lập ở một môi trường mới bất kỳ đâu: `uv sync` từ một bản clone sạch tải đúng Python 3.12 và đúng các phiên bản trong `uv.lock`.
- Một công cụ duy nhất, `uv`, với một định dạng đánh dấu (markup) thân thiện, `toml`.

`pyproject.toml` *[5]* khai báo tên và phiên bản của công cụ, bốn thư viện lúc chạy (selenium, beautifulsoup4, requests, pyyaml), và hai *runner* dưới dạng console script, tức hai lệnh `skills-scraper` và `skills-scraper-tui` xuất hiện trong `PATH` sau khi cài đặt.

```toml
[project]
name = "skills-scraper"
version = "2.0.1"
description = "Scrape Google Skills paths, courses and labs into JSON and an Obsidian-
ready Markdown vault."
readme = "README.md"
requires-python = ">=3.12"
dependencies = [
    "selenium>=4.40",
    "beautifulsoup4>=4.12",
    "requests>=2.32",
    "pyyaml>=6.0.3",
```

Mã nguồn 3: `pyproject.toml` – Tên, Phiên Bản, Các Thư Viện

```toml

[project.scripts]
skills-scraper = "skills_scraper.cli:main"
skills-scraper-tui = "skills_scraper.tui:main"

[build-system]
requires = ["uv_build>=0.8,<0.13"]
build-backend = "uv_build"
```

Mã nguồn 4: `pyproject.toml` – Hai Runner và Build Toolkits

`.python-version` cho `uv` biết phiên bản Python được dùng; `__init__.py` của gói đọc lại phiên bản từ metadata đã cài, nên con số này chỉ được khai ở một chỗ.

```
3.12

```

Mã nguồn 5: `.python-version`: Phiên Bản Python Tối Thiểu

```python
"""Google Skills Scraper: paths, courses and labs from skills.google into JSON
and Markdown."""

from importlib.metadata import version

__version__ = version("skills-scraper")
```

Mã nguồn 6: `__init__.py`: Phiên Bản Đọc Từ Metadata Của Gói

3.2. Package Và Module

Mã nguồn nằm trong một package `skills_scraper` dưới `src/`, cách bố trí được Python Packaging User Guide khuyến nghị để mã chỉ chạy được khi đã cài, không chạy nhầm từ thư mục làm việc. Package có ba sub-package và ba module ở gốc:

- `model`: cốt lõi, các lớp thực thể và tập hợp (Collection); là nơi biểu hiện rõ nhất của nội dung môn học.
- `services`: hai dịch vụ mà `model` cần, `browser.py` chạy trình duyệt; `store.py` ghi và đọc file.
- `utils`: tiện ích văn bản, ví dụ lọc ký tự không hợp lệ cho tên file và bỏ thẻ HTML.
- `config.py`, `cli.py`, `tui.py`: cấu hình và hai runner.

```
src/skills_scraper/
├── __init__.py           # __version__ đọc từ pyproject
├── config.py             # defaults < yaml < env
├── cli.py                # cli (Command Line Interface)
├── tui.py                # tui (Text User Interface)
├── model/                # LÕI QUAN TRỌNG
│   ├── __init__.py       # __init__
│   ├── base_entity.py
│   ├── collection.py
│   ├── path.py
│   ├── course.py
│   ├── courses.py
│   ├── lab.py
│   ├── labs.py
│   ├── paths.py
│   ├── serialize.py
│   └── topics.py
├── services/
│   ├── __init__.py       # __init__
│   ├── browser.py        # browser (Selenium)
│   └── store.py          # store (JSON, index, atomic)
└── utils/
    ├── __init__.py       # __init__
    └── utils.py

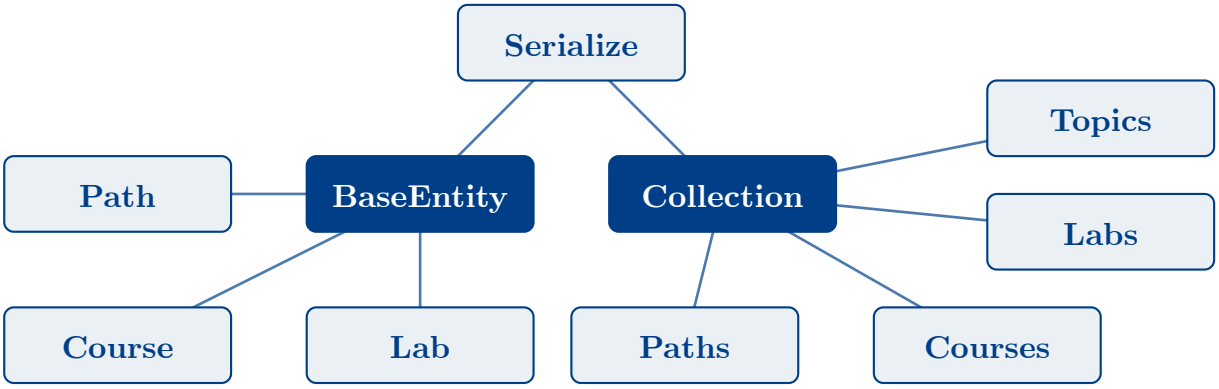
```

Mã nguồn 7: Cây Package `src/skills_scraper/`

3.3. Mô Hình Hướng Đối Tượng

3.3.1. Hai Cây Kế Thừa

Hai nhánh kế thừa xuất phát từ cùng một điểm, `Serialize`, là lớp biết biến một đối tượng thành dạng thân thiện với JSON.



Hình ảnh 3: `model/` – `Serialize` và Sơ Đồ Lớp

Trong đó:

- Trái: **thực thể** x cụ thể, ở đây là Path, Course, Lab.
  - `BaseEntity` biết id, tên, mô tả, đường dẫn file và cách ghi JSON, Markdown. Lớp con chỉ thêm dữ liệu riêng và cách render.
- Phải: **danh sách của** x, ở đây là tập hợp của Path, Course, Lab.
  - `Collection` là `{id: name}` cộng `fetch_catalog` (kéo danh sách của một loại).
  - Lớp con chỉ khai `API_URL` (danh sách các Path/Course/Lab có các URL khác nhau).
- **Kết hợp**:
  - `Portal` cho biết các `Path` khả dụng (không triển khai ở bản IE221).
  - `Path` cho biết danh sách các khoá học (`Course`).
  - `Course` tạo `Lab` và cập nhật `Labs`.
  - `Topics` đọc thuộc tính của `Courses`, `Path`, `Lab`.

`Serialize` thực ra rất đơn giản:

- `to_dict`: chuyển đối tượng thành từ điển, vốn thân thiện với JSON.
- `to_json`: chuỗi JSON của từ điển đó, chức năng cốt lõi mà các lớp kế thừa dùng trực tiếp.

```python
import json

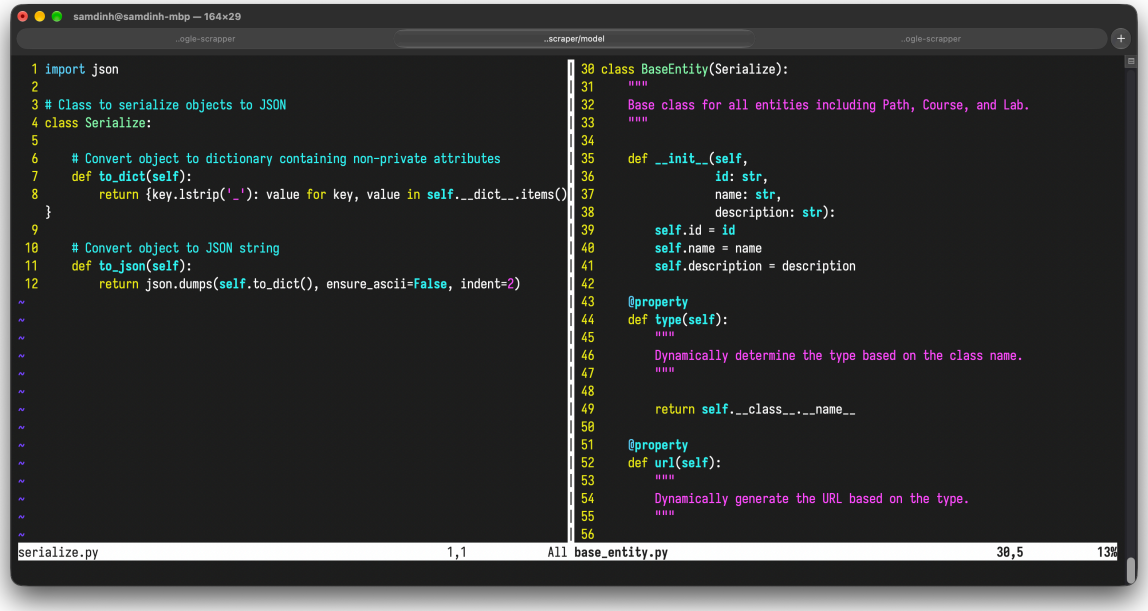
# Class to serialize objects to JSON
class Serialize:

    # Convert object to dictionary containing non-private attributes
    def to_dict(self):
        return {key.lstrip('_'): value for key, value in self.__dict__.items()}

    # Convert object to JSON string
    def to_json(self):
        return json.dumps(self.to_dict(), ensure_ascii=False, indent=2)

```

Mã nguồn 8: `model/serialize.py`: Gốc Của Cây Kế Thừa



Hình ảnh 4: Hai Cây Kế Thừa Trong Mã Nguồn

3.3.2. Lớp, Đối Tượng Và Thuộc Tính Suy Diễn

`BaseEntity.__init__()` khai báo các thuộc tính nguyên bản (attributes) để định danh một thực thể: `id`, `name`, `description`.

```
    def __init__(self,
                 id: str,
                 name: str,
                 description: str):
        self.id = id
        self.name = name
        self.description = description

```

Mã nguồn 9: `BaseEntity.__init__()` – Ba Thuộc Tính Định Danh

Nhưng để tự nhận diện và phân loại các thực thể, `BaseEntity` khai báo các thuộc tính nội sinh (properties) với `@property`, nhờ đó việc kế thừa trở nên xuyên suốt hơn:

- `@property type` trả `self.__class__.__name__`.
- `@property url` tra bảng theo `type`, trả lại `ValueError` khi gặp một loại lạ.

```python
    @property
    def type(self):
        """
        Dynamically determine the type based on the class name.
        """

        return self.__class__.__name__

    @property
    def url(self):
        """
        Dynamically generate the URL based on the type.
        """

        base_url = {
            "Path": BASE_URL_PATHS,
            "Course": BASE_URL_COURSES,
            "Lab": BASE_URL_LAB
        }.get(self.type, None)

        if not base_url:
            raise ValueError(f"Invalid entity type: {self.type}")

        return f"{base_url}/{self.id}"

```

Mã nguồn 10: `BaseEntity.type` và `BaseEntity.url`

> **Hệ Quả**
>
> Loại (`type`) của một thực thể không cần lưu sẵn mà tự suy ra, nên lớp con không thể thiếu nó; từ đó frontmatter theo chuẩn OKF luôn có `type` (gần như) miễn phí, không cần kiểm lại.

3.3.3. Kế Thừa Và Đa Hình

Kế Thừa và Đa Hình xuất hiện ở nhiều nơi trong mã nguồn, ví dụ là cặp `generate_markdown` và `save_markdown`:

- `generate_markdown`: có bản mặc định ở lớp cha và được ghi đè (override) ở Path, Course, Lab.
- `save_markdown`: ở lớp cha gọi `self.generate_markdown(**kwargs)` mà không cần biết lớp con nào đang chạy (*template method*).

```python
    def generate_markdown(self, **kwargs) -> str:
        """
        Generate the Markdown representation of the entity.
        """

        # Convert the Path object to a dictionary
        markdown = []
        markdown.append(self.generate_front_matter())

        return "\n\n".join(markdown) + "\n"
```

Mã nguồn 11: `BaseEntity.generate_markdown` – Mặc Định Chỉ Có Frontmatter

```
    def save_markdown(self, **kwargs) -> None:
        """
        Save the Path data to a Markdown file.
        passes kwargs to generate_markdown
        """

        write_text(self._md_path, self.generate_markdown(**kwargs))

```

Mã nguồn 12: `BaseEntity.save_markdown`: gọi `generate_markdown` Của Lớp Con

3.3.4. generate_markdown

`model/lab.py` là lớp con nhỏ nhất, đủ để thấy toàn bộ cơ chế: `__init__` gọi `super()`, thêm hai thuộc tính riêng, rồi ghi đè `generate_markdown`.

`Lab` kế thừa `BaseEntity`:

```
class Lab(BaseEntity):
    """
    Class representing a Lab entity.
    """

    def __init__(self,
                 id: str,
                 name: str = None,
                 description: str = None,
                 steps: dict = None):
        super().__init__(id,
                         name,
                         description
                         )
        self.steps = steps or {}
```

Mã nguồn 13: `model/lab.py` – Kế Thừa `BaseEntity`

Và ghi đè `generate_markdown`:

```
    def generate_markdown(self, toc_only: bool = False, **kwargs):
        """
        Generate the Markdown content for the Lab entity.
        :param toc_only: If True, only generate table of contents (structure).
        """

        markdown = []
        markdown.append(self.generate_front_matter())

        markdown.append(f"# [{self.name}]({self.url})")

        if not toc_only:
            if hasattr(self, 'description') and self.description:
                markdown.append(f"{self.description}")

        if hasattr(self, 'steps') and self.steps:
            for step_number, step_text in self.steps.items():
                markdown.append(f"## Step {step_number}: {step_text}")

        return "\n\n".join(markdown) + "\n"

```

Mã nguồn 14: `model/lab.py` – Ghi Đè `generate_markdown`

3.3.5. fetch_catalog

Cùng cơ chế đó ở nhánh bên phải: `Collection.fetch_catalog()` là một bản duy nhất, đọc `API_URL` và `MAX_PAGES` là thuộc tính lớp; `Paths`, `Courses`, `Labs` chỉ khai lại `API_URL`. Ở v2.0.0 có ba bản copy của vòng lặp phân trang này, mỗi lớp một bản.

```python
class Courses(Collection):
    """
    Class representing a collection of courses.
    """

    API_URL = API_URL_COURSES

    def __init__(self,
                 name: str = None,
                 url: str = BASE_URL_COURSES,
                 collection: dict = None,
                 driver=None):
        super().__init__(name, url, collection, driver)

    def fetch_courses(self, force: bool = False) -> bool:
        """
        Gather all courses from the catalog API. Returns a Boolean to check status.
        """
        return self.fetch_catalog(force=force)

```

Mã nguồn 15: `model/courses.py` – Lớp Con Chỉ Khai `API_URL`

3.3.6. Đóng Gói Và Kết Hợp

3.3.6.1. Đóng Gói

- Thuộc tính "ẩn" và động: `_json_path`, `_md_path`, `_kind` là property bắt đầu bằng `_`, suy ra từ `type` và `id`.
- `to_dict()` lọc khoá bắt đầu bằng `_` và khoá `driver`, nên JSON ghi ra không chứa tài nguyên hay đường dẫn máy.
  - `driver` là tài nguyên (một cửa sổ Chrome), không phải thuộc tính của thực thể, không cần lưu giữ.
- Các thuộc tính theo dõi như `scrapedTime` được thêm vào lúc ghi file JSON.

```
        the_dict = {k: v for k, v in self.__dict__.items() if not k.startswith('_') and
k != 'driver'}
        the_dict['type'] = self.type
        the_dict['url'] = self.url

        # Add scrapedTime (epoch in milliseconds)
        the_dict['scrapedTime'] = int(time.time() * 1000)

        return the_dict
```

Mã nguồn 16: `BaseEntity.to_dict()` – Lưu Gì Ra File JSON

3.3.6.2. Kết Hợp (Composition)

`Course.process_lab()` là chỗ một course tạo ra một lab và lưu lab đó, đồng thời cập nhật tập hợp `Labs`:

- `lab.load_json()` nạp file JSON của lab nếu đã có; đã có nghĩa là đã tải, bỏ qua để giữ tính nguyên vẹn như thế (idempotent).

```
            lab = Lab(
                id=lab_id
            )
            # Load the lab data from JSON even it's empty
            lab.load_json()

            # If the lab.name does exist, that means the lab has been extracted already.
            if lab.name:
                print(f"(process_lab) •-• [+] Existed: {lab.id} - {lab.name}")
                return False
```

Mã nguồn 17: `Course.process_lab()` – Tạo `Lab`, Nạp JSON

- Nếu chưa có, tải lab về rồi `lab.save_json()` cùng `lab.save_markdown()`.

```
            # Save the lab to files.
            lab.save_json()
            lab.save_markdown()
```

Mã nguồn 18: `Course.process_lab()` – Lưu `Lab` Vừa Tải

- Đồng thời, đưa lab vào danh sách các lab đã biết.

```
            # Add the lab to the Labs Collection
            labs_collection = Labs(name='Labs Collection')
            labs_collection.load_json()
            labs_collection.collection[lab_id] = lab.name
            labs_collection.save_json()
```

Mã nguồn 19: `Course.process_lab()` – Cập Nhật Tập Hợp `Labs`

3.4. Kiểu Dữ Liệu, Hàm Và Cấu Trúc Điều Khiển

3.4.1. Kiểu Dữ Liệu

Dữ liệu của một `Course` là `dict` (từ điển) lồng ba tầng: `modules` chứa `steps` chứa `activities`. Mỗi tầng là `list` (danh sách) của `dict`; đây cũng chính là hình dạng của file JSON, nên `json.loads` và `json.dumps` là nạp và xuất dữ liệu trực tiếp không cần lớp chuyển đổi.

```python
class Course(BaseEntity):
    def __init__(self,
                 id: str,
                 name: str = None,
                 description: str = None,
                 datePublished: str = None,
                 objectives: list = None,
                 topics: list = None,
                 modules: list = None,
                 driver=None):
        super().__init__(id,
                         name,
                         description)
        self.datePublished = datePublished or ""
        self.objectives = objectives or []
        self.topics = topics or []
        self.modules = modules or []
        self.driver = driver
        self.external_course_data = {}

```

Mã nguồn 20: `Course.__init__()` – Kiểu Dữ Liệu

3.4.2. Cấu Trúc Điều Khiển

Rẽ nhánh bằng `dict` (data-driven) thay cho chuỗi `if/elif`: `Course.process_step()` tra bảng `handlers` theo `activity['type']`.

- Gặp loại dữ liệu lạ (ví dụ Google mới thêm `badge`, `credential`) thì giữ nguyên dữ liệu thô, không xử lý và in thông báo.
    - Đây là chỗ thể hiện sự "trung thực" về hành vi và sự tham gia của con người: tường minh, có giới hạn, không phỏng đoán.

```python
    def process_step(self, step) -> None:
        """
        Process each step in a module.
        """

        # One handler per activity kind. The set of kinds belongs to the site, not to us:
        # a kind we have never seen (badge, credential, ...) is kept as-is and logged.
        handlers = {
            "video": self.process_video,
            "lab": self.process_lab,
            "quiz": self.process_quiz,
            "link": self.process_link,
            "html_bundle": self.process_link,
            "document": self.process_document,
        }
```

Mã nguồn 21: `Course.process_step()` – Bảng `handlers`

```
        for activity in step['activities']:
            # Fix HREF if it is a session URL
            if activity.get('href') and '/course_sessions/' in activity['href']:
                activity['href'] = re.sub(r'/course_sessions/[^/]+', f'/
course_templates/{self.id}', activity['href'])

            activity_type = activity['type']
            activity_full_url = f"{BASE_URL}{activity['href']}"

            handler = handlers.get(activity_type)
            if handler:
                handler(activity, activity_full_url)
            else:
                print(f"(process_step) -> {activity_type}: {activity['id']:>6} -
{activity['title']} (kept as-is, no handler)")
```

Mã nguồn 22: `Course.process_step()` – Loại Lạ Được Giữ Nguyên và Thông Báo

3.4.3. Vòng Lặp Và Hàm

Đệ quy, vì `items` của một bài học có thể lồng nhau: `Course._parse_lesson_item()` xử lý một item rồi gọi chính nó cho các item con.

```
    def _parse_lesson_item(self, item: dict) -> str:
        """
        Parse a single item from the lesson data into Markdown/HTML.
        """
        # Based on observed structure, items have 'heading', 'paragraph', 'list', etc.
        # This is a best-effort parser.

        # Check for nested items (some structures might be recursive or wrapped)
        if 'items' in item:
            sub_content = []
            for sub in item['items']:
                parsed = self._parse_lesson_item(sub)
                if parsed:
                    sub_content.append(parsed)
            return "\n\n".join(sub_content)
```

Mã nguồn 23: `Course._parse_lesson_item()`: Lặp Đệ Quy Trên Cây *item*

Lặp có ngắt: `Collection.fetch_catalog()` phân trang bằng `while True` với một "điểm ngắt" `MAX_PAGES`, dừng khi trang trả về rỗng hoặc vượt giới hạn; dùng khi lấy danh sách toàn bộ Path, Course, Lab có trên skills.google.

```
    def fetch_catalog(self, force: bool = False) -> bool:
        """
        Gather every item of this collection from the paged catalog API,
        read through the signed-in browser (the JSON lands in a <pre> tag).
        Returns True on success.
```

Mã nguồn 24: `Collection.fetch_catalog()` – Bắt Đầu

```
                for item in items:
                    title = item.get("title")
                    item_path = item.get("path")
                    if title and item_path:
                        # /paths/16?locale=en -> 16 : drop the query, keep the last
segment
                        item_id = item_path.split('?')[0].split('/')[-1]
                        if item_id:
                            items_found[item_id] = title.strip()

                page += 1
                if page > self.MAX_PAGES:
                    print(f"\nReached safety limit of {self.MAX_PAGES} pages.")
                    break

            print(f"\nTotal {label} found: {len(items_found)}")
```

Mã nguồn 25: `Collection.fetch_catalog()` – Vòng Lặp Có Điểm Ngắt

3.5. Xử Lý Ngoại Lệ

> **Nguyên Tắc**
>
> Một *activity* hỏng không làm hỏng cả *course* trong quá trình `fetch`; một ngắt giữa chừng không để lại nửa file; mọi lỗi đều được thông báo rõ ràng, không có lỗi im lặng.

Các ngoại lệ gặp trong mã thuộc ba nhóm:

- Của thư viện: `NoSuchElementException` (Selenium) khi trang thiếu phần tử selector được mong đợi; `json.JSONDecodeError` khi danh mục không phải JSON; `OSError` khi ghi file.
- Của Python: `KeyboardInterrupt` và `EOFError` khi người dùng dừng ở bước đăng nhập; `KeyError` khi tra bảng trình duyệt; `ValueError` khi `type` lạ.
- `try` và `catch` có chủ ý: mỗi `process_*` của một activity nằm trong `try/except Exception` riêng và in lỗi kèm tên phương thức, để course đi tiếp sang activity kế.

`services/store.py` ghi file nguyên tử (atomic): ghi vào file tạm cùng thư mục, `fsync`, rồi `os.replace`; nếu có lỗi hoặc Ctrl+C giữa chừng thì xoá file tạm và trả lại ngoại lệ, không bao giờ để lại "nửa" file.

```
def write_text(path, text: str) -> None:
    """
    Write UTF-8 text atomically.
    """
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    fd, tmp = tempfile.mkstemp(dir=path.parent, prefix=".tmp-")
    try:
        with os.fdopen(fd, "w", encoding="utf-8", newline="\n") as textfile:
            textfile.write(text)
            textfile.flush()
            os.fsync(textfile.fileno())
        os.replace(tmp, path)
    except BaseException:
        try:
            os.remove(tmp)
        except OSError:
            pass
        raise

```

Mã nguồn 26: `store.write_text()` – Ghi Nguyên Tử Với `try/except/raise`

`services/browser.py` xử lý trường hợp người dùng bỏ dở thao tác đăng nhập: `input()` bị ngắt bằng Ctrl+C hay Ctrl+D được bắt và trả về `False`, thông báo rõ ràng.

```
    while "sign_in" in driver.current_url:
        suffix = f" for {what}" if what else ""
        print(f"\n\033[93m[!] Authentication required{suffix}.\033[0m")
        print("Please sign in to the browser window if you haven't.")
        try:
            input("Press Enter after you have signed in and the page is loaded... ")
        except (KeyboardInterrupt, EOFError):
            print("\n\033[91mAborted authentication.\033[0m")
            return False
        driver.get(url)

        if "sign_in" in driver.current_url:
            print("\n\033[91m[!] Still on the sign-in page. Finish signing in before
pressing Enter.\033[0m")
    return True
```

Mã nguồn 27: `browser.ensure_authenticated()` – `KeyboardInterrupt`, `EOFError`

`BaseEntity.save_json` có `OSError` của bước ghi và chỉ cập nhật chỉ mục khi ghi xong, nên chỉ mục không bao giờ trỏ tới một file không tồn tại.

```python
    def save_json(self):
        """
        Write the entity to its JSON file (atomically) and update the index.
        """
        entity_data = self.to_dict()
        try:
            write_json(self._json_path, entity_data)
        except OSError as e:
            print(f"(BaseEntity.save_json) Error writing JSON to file:
{self._json_path}")
            print(e)
            return

        index = Index()
        index.upsert(self._kind, entity_data)
        index.save()
```

Mã nguồn 28: `BaseEntity.save_json()` – Ghi File Trước, Cập Nhật Chỉ Mục Sau

3.6. Phiên Đăng Nhập Và Thu Thập

Từ đầu 2026, trang Course trên skills.google chuyển hướng về trang đăng nhập nếu chưa đăng nhập; vì vậy mọi trang đều đi qua một trình duyệt Chrome (hoặc Edge) do Selenium *[6]* điều khiển, `requests` chỉ còn dùng để tải tài liệu PDF với cookie lấy từ trình duyệt. Thay đổi này được cập nhật ở phiên bản v2.0.0 của nhánh chính (*master*) trong kho git.

Toàn bộ việc này nằm trong `services/browser.py`, với ba hàm công khai:

- `launch_browser()` mở trình duyệt trên một *profile* cố định (`.webdriver_profiles` theo mặc định), nên phiên đăng nhập được giữ qua các lần chạy: đăng nhập một lần, những lần sau mở trực tiếp. Một danh sách cờ dùng chung cho Chrome và Edge.
- `ensure_authenticated()` chờ người dùng: nếu URL hiện tại chứa `sign_in` thì in hướng dẫn, chờ Enter, tải lại trang, lặp tới khi hoàn thành hoặc người dùng hủy bỏ.
- `get_page()` gói hai bước trên và trả về cây `BeautifulSoup` *[7]* của trang đã render; `model/` chỉ gọi hàm này, không (cần) biết Selenium.

```
    try:
        options_class, service_class, driver_class = BROWSERS[browser.lower()]
    except KeyError:
        raise ValueError(f"Unsupported browser: {browser} (chrome or edge)")

    options = options_class()
    for argument in BROWSER_ARGUMENTS:
        options.add_argument(argument)
    options.add_experimental_option("excludeSwitches", ["enable-logging", "enable-
automation"])

    # The "new" headless mode is the one that behaves like a real window.
    if headless:
        options.add_argument("--headless=new")

    if profile_folder:
        options.add_argument(f"user-data-dir={os.path.abspath(profile_folder)}")

    return driver_class(service=service_class(), options=options)
```

Mã nguồn 29: `browser.launch_browser()` – Khởi Chạy Trình Duyệt

```
def open_page(driver, url: str, what: str = "") -> bool:
    """
    Navigate the browser to url, dealing with the sign-in redirect.
    Returns False when there is no browser or the user aborted the sign-in.
    """
    if not driver:
        print(f"(open_page) A signed-in browser is required{' for ' + what if what else
''}.")
        return False
    driver.get(url)
    return ensure_authenticated(driver, url, what)

def get_page(driver, url: str, what: str = "") -> BeautifulSoup | None:
    """
    open_page(), then hand back the rendered page as BeautifulSoup.
    """
    if not open_page(driver, url, what):
        return None
    return BeautifulSoup(driver.page_source, "html.parser")
```

Mã nguồn 30: `browser.open_page()`, `browser.get_page()` – Duyệt Trang

Việc thu thập một course (`Course.extract_transcript()`) đi theo bốn bước, mỗi bước là một phương thức:

1. `extract_course_metadata()` đọc khối `ld+json` của trang: `@id`, `name`, `datePublished`, `about` (topics), `teaches` (objectives); mô tả lấy từ `meta[name=description]`.
2. `extract_course_outline()` đọc đề cương từ thẻ `ql-contents-menu`, cho ra cây `modules` → `steps` → `activities`.
3. `process_step()` cho từng step, xử lý theo loại activity (Mã nguồn 21): video thì lấy transcript YouTube, lab thì tạo `Lab`, quiz thì lấy câu hỏi, document thì tải file, link và html_bundle thì giữ URL.
4. `save_json()` để lưu file JSON rồi `save_markdown()` để lưu file Markdown.

Quy tắc bỏ qua nằm ở bước đầu: nếu `datePublished` trên trang bằng giá trị đã lưu trong JSON và không có `--force` trong lệnh gọi, phương thức in lý do và trả về `False`, `course` dừng tại đó; đây là FR-05: nếu chạy lại trên cùng một Path/Course/Lab đã tồn tại thì kết quả vẫn là nguyên bản (idempotent).

```
            course_objectives_json = json.loads(course_ld_json_text)
            datePublished = course_objectives_json.get('datePublished')

            # If the course has the same datePublished, return False and not continue.
            if not force and datePublished == self.datePublished:
                print(f"(extract_course_metadata) Course {self.id} already extracted.
datePublished: {datePublished}\n")
                return False

            self.id = course_objectives_json.get('@id').split('/')[-1]
            self.name = course_objectives_json.get('name').strip()
            self.description = course_description
            self.datePublished = course_objectives_json.get('datePublished')
            self.topics = course_objectives_json.get('about')
            self.objectives = course_objectives_json.get('teaches')
```

Mã nguồn 31: `Course.extract_course_metadata()` – Bỏ Qua Theo `datePublished`

Với path, `Path.fetch_data()` đọc `ql-contents-menu[modules]` của trang path làm nguồn sự thật, vì `href` ở đó phân biệt `/course_templates/<id>` (course) với `/focuses/<id>` (lab), còn `ld+json.hasPart` chỉ dùng để bổ sung tên và mô tả; `fetch -p ID --cascade` sau đó lần lượt thu thập từng course trong path.

3.7. Lưu Trữ

Không có cơ sở dữ liệu. Mỗi thực thể là một file JSON và file đó là nguồn sự thật (SSOT); mọi thứ khác đều dẫn xuất và dựng lại được.

```
data/
├── index.json          chỉ mục: {kind: {id: {name, datePublished, scrapedTime}}}
├── paths.json          tập hợp {id: name} của Paths (Collection)
├── paths/280.json      một path có id 280
├── courses/892.json    một course có id 892, đầy đủ modules/steps/activities
└── labs/<id>.json      một lab

```

Mã nguồn 32: `data/` – Mỗi Thực Thể Một File

`services/store.py` cung cấp đúng những gì `model/` cần:

- `write_text()`, `write_json()`, `read_json()`: đọc ghi UTF-8, ghi luôn nguyên tử (Mã nguồn 26).

```python
def write_text(path, text: str) -> None:
    """
    Write UTF-8 text atomically.
    """
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    fd, tmp = tempfile.mkstemp(dir=path.parent, prefix=".tmp-")
    try:
        with os.fdopen(fd, "w", encoding="utf-8", newline="\n") as textfile:
            textfile.write(text)
            textfile.flush()
            os.fsync(textfile.fileno())
        os.replace(tmp, path)
    except BaseException:
        try:
            os.remove(tmp)
        except OSError:
            pass
        raise

```

Mã nguồn 33: `store.write_text()` – Lưu File

3.8. Kết Xuất Markdown

Lệnh `md` (và bước cuối của `fetch`) biến JSON thành note Markdown cho Obsidian, hoàn toàn offline. Mỗi note gồm frontmatter YAML và thân bài.

Frontmatter do `BaseEntity.generate_front_matter()` sinh, là tập mở rộng (superset) của OKF v0.2 *[3]*: các khoá OKF (`type`, `title`, `description`, `resource`, `tags`, `sources`, `generated`), hai khoá riêng của công cụ (`id`, `portal`), và các tên cũ mà vault hiện có đang truy vấn (`url`, `date_published`, `topics`, `scraped_date`).

Kết quả thật cho course 892 từ `docs/samples/data/` trong repo:

Tạo file bằng:

```bash
uv run skills-scraper md -c 892 --toc
```

Kết quả:

```yaml
---
id: "892"
title: "Conversational AI on Vertex AI and Dialogflow CX"
type: Course
portal: public
description: "In this course you will learn how to use the new generative AI features in
Dialogflow CX ..."
resource: https://www.skills.google/course_templates/892
sources:
  - resource: https://www.skills.google/course_templates/892
    title: "Google Skills course 892"
    author: Google Cloud
    last_modified: 2023-11-21
generated:
  by: skills-scraper/2.0.1
  at: 2026-10-16T14:16:02+00:00
url: https://www.skills.google/course_templates/892
date_published: 2023-11-21
topics: []
scraped_date: 2026-10-16
---
```

Mã nguồn 34: `892.md` – Frontmatter Từ JSON

```python
        lines = ["---"]
        lines.append(f"id: {yaml_scalar(self.id)}")
        lines.append(f"title: {yaml_scalar(self.name)}")
        lines.append(f"type: {self.type}")
        lines.append("portal: public")
        if description:
            lines.append(f"description: {yaml_scalar(first_sentence(description))}")
        lines.append(f"resource: {self.url}")
        if topics:
            lines.append(f"tags: {yaml_list(topics)}")
        lines.append("sources:")
        lines.append(f"  - resource: {self.url}")
        lines.append(f"    title: {yaml_scalar(f'Google Skills {self.type.lower()}
{self.id}')}")
        lines.append("    author: Google Cloud")
        if date_published:
            lines.append(f"    last_modified: {date_published}")
        lines.append("generated:")
        lines.append(f"  by: skills-scraper/{__version__}")
        lines.append(f"  at: {scraped_at.isoformat(timespec='seconds')}")
        # Legacy names, kept so existing vault queries keep working.
        lines.append(f"url: {self.url}")
        if date_published:
            lines.append(f"date_published: {date_published}")
        if hasattr(self, 'topics'):
            lines.append(f"topics: {yaml_list(topics)}")
        lines.append(f"scraped_date: {scraped_at.strftime('%Y-%m-%d')}")
        lines.append("---")
        return "\n".join(lines)
```

Mã nguồn 35: `BaseEntity.generate_front_matter()` – Tạo Frontmatter

Quy ước cấu trúc nội dung file Markdown:

- Tiêu đề cấp 1 là tên course có liên kết về trang gốc trên Google Cloud.
- Mỗi module một tiêu đề cấp 2.
- Mỗi activity một tiêu đề cấp 3 có loại và liên kết
  - Video kèm liên kết YouTube và transcript (không sinh transcript khi gọi lệnh `md` với `--toc` hoặc `--no-transcript`)

    - Quiz là callout với từng câu hỏi.
    - Tài liệu PDF được tải vào `<vault>/materials/courses/<id>/` và liên kết tương đối từ MD.
- MD của path liệt kê các course và lab của nó bằng liên kết tương đối `../courses/...`, `../labs/...`
    - Chính các liên kết này tạo nên đồ thị ở Hình ảnh 1 mà không cần thiết kế trước.

3.9. Hai Runner

Cả hai runner đều chỉ `import` từ `model/` và `services/`; không có logic nghiệp vụ nào trong `cli.py` hay `tui.py`. Thêm một runner thứ ba bất kỳ (chẳng hạn một HTTP API) sẽ không chạm/thay đổi/ảnh hưởng tới `model/`.

3.9.1. Giao Diện Dòng Lệnh

`cli.py`:

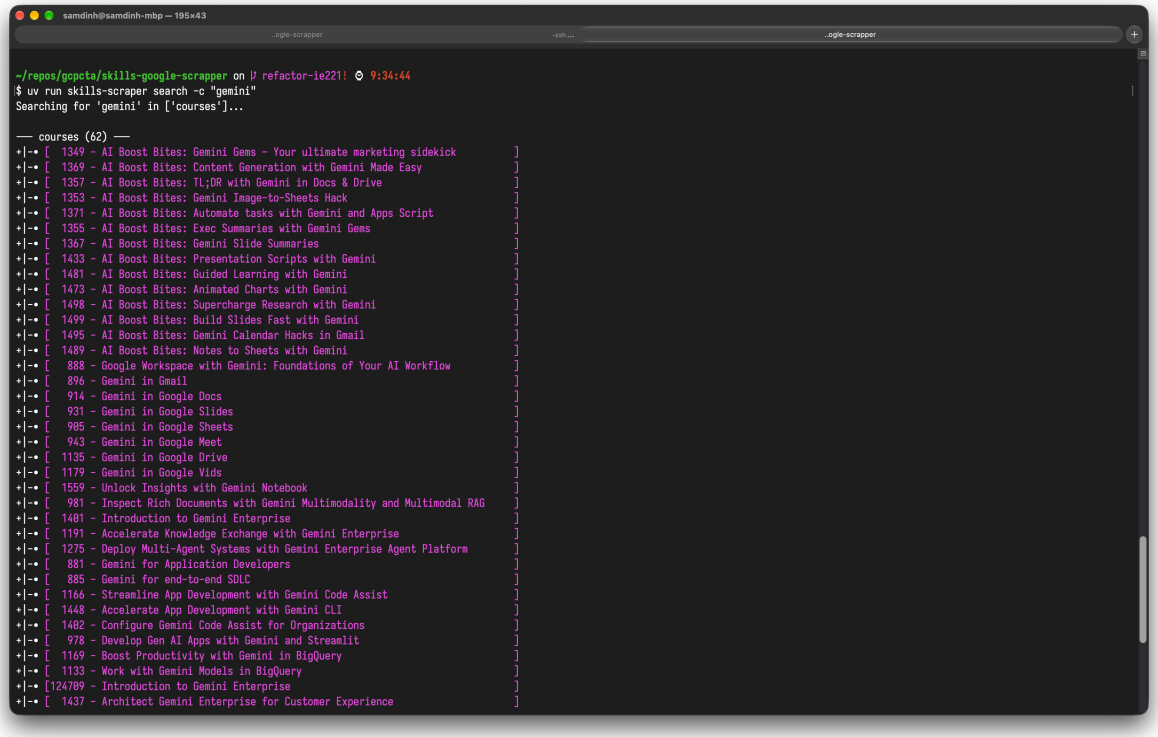
Lệnh	Làm gì	Cờ chính
`list (l)`	Liệt kê path, course hoặc lab từ `data/`	`-p` / `-c` / `-l`, `-r` làm mới từ site, `-i` sắp theo id, `--headless`
`fetch (f)`	Thu thập vào `data/` và sinh Markdown	`-p ID...` / `-c ID...` / `-l ID...`, `--cascade`, `--force`, `--toc`, `--no-transcript`, `--no-md`
`md`	Sinh Markdown từ JSON đã có, offline	`-c ID,ID` / `-p ID` / `-l ID`, `--toc`, `--no-transcript`
`search (s)`	Tìm trong dữ liệu đã tải	`QUERY`, `-c` / `-p` / `-l`, `--field name|topics|...`
`browser (b)`	Mở trình duyệt đã đăng nhập và chờ, để đăng nhập	`--profile-folder`

Bảng 7: `skills-scraper – cli`

Vì giao tiếp bằng file và mã thoát, CLI ghép được với shell và các công cụ khác mà không cần API riêng, ví dụ thu thập mọi course của một path rồi xem site đã đổi gì so với lần trước:

```
uv run skills-scraper fetch -p 280 --cascade --toc
jq -r '.courses | keys[]' data/paths/280.json | while read id; do
  uv run skills-scraper md -c "$id" --toc
done
git -C csbmdvault diff --stat
```

Mã nguồn 36: CLI – Kết Hợp Với `jq`, `while` của Shell và `git`



Hình ảnh 5: `skills-scraper` Trong Terminal

3.9.2. Giao Diện Tương Tác

`tui.py`:

- Menu số và chữ.
- Màu ANSI.
- Vòng lặp `while` đọc `input()` và rẽ nhánh theo lựa chọn:
  - `c` một course
  - `p` một path
  - `l` danh sách
  - `h` tất cả course
  - `w` mở trình duyệt
  - `q` thoát.

Trình duyệt dùng chung: `CloudSkillsBoost.driver` là một `@property` khởi tạo lười (lazy), chỉ mở Chrome khi lựa chọn đầu tiên cần, và `close()` đóng nó trong khối `finally` của `main()`, nên thoát bằng `q` hay Ctrl+C đều không để lại tiến trình Chrome mồ côi (orphaned/zoombie).

```python
    @property
    def driver(self):
        """The signed-in browser, launched on first use and reused after that."""
        if self._driver is None:
            print("\n\033[35mLaunching the browser...\033[0m\n")
            self._driver = launch_browser()
        return self._driver

    def close(self):
        """Quit the browser if one was launched."""
        if self._driver is not None:
            self._driver.quit()
            self._driver = None

```

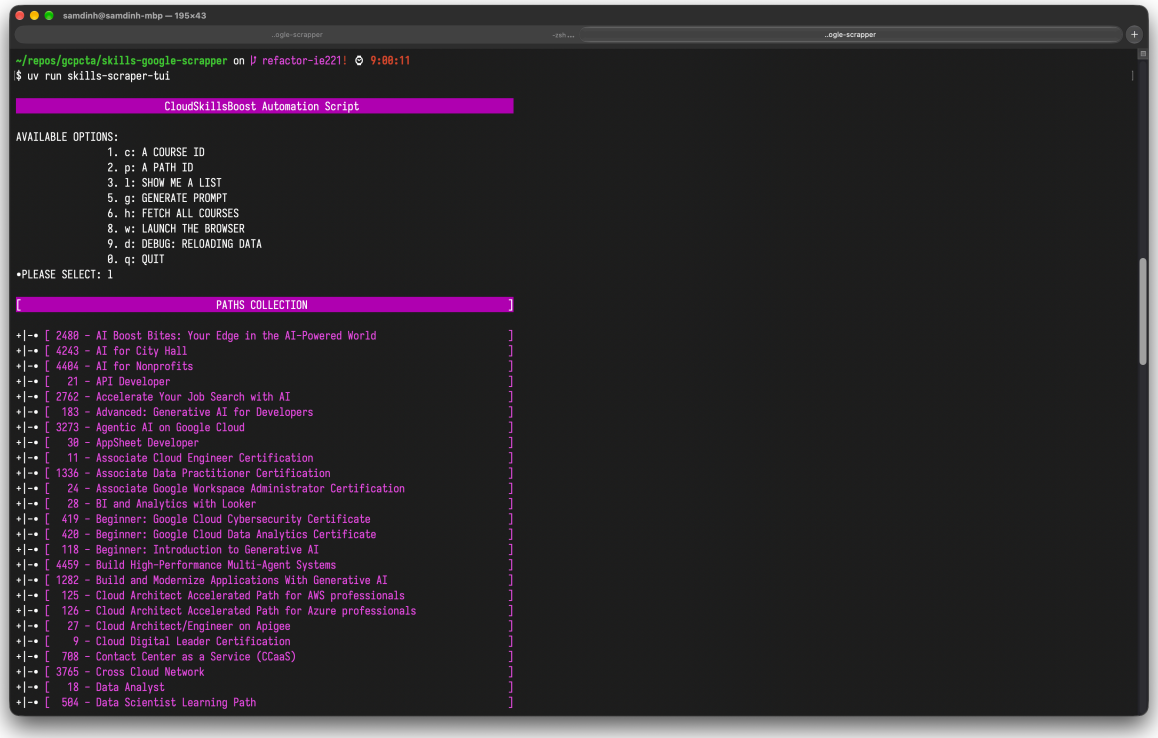
Mã nguồn 37: `CloudSkillsBoost.driver()` – Khởi Tạo Trình Duyệt

```python
    print("\033[45m"
          f"{app_title:^87}"
          "\033[0m")
    print()

    # Create an instance of CloudSkillsBoost and start interactive mode
    cloud_skills_boost = CloudSkillsBoost()
    try:
        cloud_skills_boost.interactive_mode()
    finally:
        cloud_skills_boost.close()

    sys.exit(0)
```

Mã nguồn 38: `tui.main` – `try/finally` – Đóng Trình Duyệt Khi Thoát



Hình ảnh 6: `skills-scraper-tui` – Cùng Lõi `model` – Khác Giao Diện

3.10. Kết Chương

Chương 3 trình bày về các nội dung kiến thức của môn học IE221 dựa trên mã nguồn đã có sẵn của kho git, đặc biệt trên branch `refactor-ie221` từ các phần cơ bản nhất như gói và module với `uv` và `src/` (Mã nguồn 7); tính kế thừa qua 2 cây kế thừa từ `Serialize` (Hình ảnh 3); kiểu dữ liệu lồng, phân nhánh bằng `dict`, đệ quy và vòng lặp có điều kiện thoát; bắt và xử lý ngoại lệ, vv...

Và từ đó, Chương 4 sẽ trả lời thêm câu hỏi: làm sao biết tất cả những điều trên vẫn đúng khi website thay đổi, vì vốn đây là điều chắc chắn có, và sẽ có cũng như ngoài tầm kiểm soát của công cụ (hay chính xác hơn là sinh viên tạo ra công cụ).

CHƯƠNG 4.

KIỂM THỬ, VẬN HÀNH VÀ BẢO TRÌ

Vốn chỉ là một công cụ (scraper) và chỉ làm đúng một việc, có nghĩa công cụ chỉ tồn tại bằng cách theo đuổi yếu tố bên ngoài: một website không thuộc về mình; nên câu hỏi quan trọng hơn cần được xử lý không phải là "có chạy được không", mà là "làm sao biết nó còn chạy đúng, và khi không còn đúng thì biết là sai ở đâu".

4.1. Kiểm Thử Offline

Mọi test chạy không cần mạng và không cần trình duyệt. Nguồn vào là các trang đã ghi lại từ website trong `tests/fixtures/google_skills/`, thực ra là các trang được lưu thủ công, và chỉ giữ lại phần mã nguồn html, ví dụ: trang course 72 (`course-72.html`), thẻ `ql-contents-menu`, cây `modules` đã phân tích (`course-72-modules.json`, vv..).

Test sẽ khẳng định dựa trên **cấu trúc** (schema) của dữ liệu mẫu tại thời điểm đó (có `datePublished`, mỗi activity có `id`, `type`, `title`, `href`), mà không phải là dựa trên **giá trị** (tiêu đề course, description, vv.. vì đây là các giá trị có thể thay đổi bất kỳ lúc nào), nên mẫu (fixture) không phải soi chiếu lại mỗi khi Google chỉ thay đổi các phần giá trị mà không thay đổi cấu trúc.

Tệp	Kiểm tra	Số test
`test_course.py`	• Metadata từ `ld+json`; bỏ qua khi `datePublished` không đổi; đề cương nội dung từ `ql-contents-menu`; Markdown render được mọi loại activity kể cả loại lạ; danh sách dùng dấu gạch	5
`test_frontmatter.py`	• Dấu nháy trong tiêu đề vẫn là YAML hợp lệ (trong YAML rất nhạy cảm với các dấu nháy đơn/kép) ; đủ khoá OKF và alias cũ; mọi loại thực thể đều có `type`.	3
`test_path.py`	• `ql-contents-menu` phân biệt lab với course; không có menu thì lùi về `ld+json`; note path liên kết lab tới `labs/`	3
`test_store.py`	• `write_json` không để lại file tạm; `upsert` giữ trường cũ; `search` theo tên rồi theo nội dung file; `rebuild` bỏ qua file prompt	4

Bảng 8: Kiểm Thử – Bộ Test Offline

Ví dụ với Python 3.12.14 do `uv` tải:

```
$ uv sync
Using CPython 3.12.14
Creating virtual environment at: .venv
Resolved 30 packages in 0.56ms
Installed 27 packages in 3ms
$ uv run pytest -q
...............
[100%]
15 passed in 0.20s
```

Mã nguồn 39: `uv sync` và `uv run pytest`

Bộ test đồng thời còn là các **cảnh báo sớm**: khi nghi vấn hoặc đã biết Google đổi markup, bước đầu tiên của người bảo trì là ghi lại trang mới vào `tests/fixtures/` (thực ra là save as và chọn lưu ở dạng html không kèm theo asset, hoặc đơn giản là mở Source và copy phần chứa các selector được đề cập ở trên); test nào báo lỗi thì đó là chỗ cần sửa, và từ đó thay đổi mã nguồn trước khi thực hiện lại trên website thật.

4.2. Chạy Lại Và Tính Idempotent

NFR-01 là một yêu cầu phi chức năng, nhưng tập trung vào tính toàn vẹn/nhất quán của dữ liệu. Yêu cầu này cho biết nếu chạy lại cùng một lệnh n lần không làm hỏng và không làm thừa (dữ liệu đã có). Ba cơ chế trong mã nguồn sau bảo đảm điều đó, mỗi cơ chế thực ra đều đã được chỉ ra ở Chương 3:

- `fetch -c 892` lần hai:
  - `extract_course_metadata` so `datePublished` trên trang với giá trị đã lưu, bằng nhau thì in `Course 892 already extracted. datePublished: 2023-11-21` và trả `False`, course dừng tại đó (Mã nguồn 31);
  - `--force` để ép thu thập lại và thay thế nội dung, không quan trọng là `datePublished` có cùng giá trị hay không.
- Lab đã có file JSON thì `process_lab` không tải lại (Mã nguồn 17):
  - Vì một lab xuất hiện trong nhiều course và nhiều path.
  - Giả sử lab 123 thuộc cả course ABC và XYZ, nếu đã được hoàn thành ở course ABC thì không ghi đè và khi duyệt tới course XYZ sẽ vẫn thấy là lab 123 đã được hoàn thành, làm lại lab hay không là quyền của người học, công cụ không nên và không được thay đổi bản chất tình huống.
- Ctrl+C giữa chừng:
  - Mọi file đi qua `write_text` nguyên tử (Mã nguồn 26).
  - Vì vậy nên trên đĩa hoặc là bản cũ nguyên vẹn hoặc là bản mới hoàn chỉnh, không bao giờ là nửa file.
  - Cơ chế kiểm tra không còn `.tmp-*` sau khi ghi, có nghĩa là việc hoàn thành ghi file không chỉ đơn giản hoàn thành ở đoạn *cuối* (file JSON được tạo ra), mà đoạn *đầu* (file `.tmp-*`) phải không còn tồn tại nữa.

- Hay nói cách khác: tại bất kỳ mọi thời điểm, luôn bảo đảm chỉ có một và một *SSOT*.

Lệnh `md` thì thuần hàm sinh file, không thay đổi (và quan tâm) dữ liệu của SSOT:

- Cùng JSON vào, cùng Markdown ra.
- Chạy bao nhiêu lần cũng được và `git diff` trên vault trả về rỗng nếu dữ liệu không đổi.
- Đây cũng là một cách phát hiện site đã đổi gì: `fetch --force` rồi `git diff` vault (nếu không `git diff` trong data/json).

4.3. Khi Nền Tảng Thay Đổi Và Hành Vi

Quy trình xử lý thay đổi của site gồm ba bước, như đã nêu ở Phần 2.2: ứng dụng phát hiện và báo (log, test thất bại, dừng rõ ràng); con người đánh giá giá trị và chi phí; thay đổi được thực hiện thành một commit có nguồn và lý do (justification): không tùy tiện, phải chỉ rõ thay đổi đó là gì, tại sao, giá trị mang lại là gì, và thời điểm nào, vv...

Bảng dưới là các thay đổi thật của skills.google từ v2.0.0 (02/2026) tới nay, cách dòng chính đã xử lý, và commit tương ứng trên `refactor-ie221`.

Thay Đổi	Triệu Chứng	Cách Xử Lý	`master`	`ie221`
Trang course bắt buộc đăng nhập	`requests.get` nhận trang sign-in, hoặc dừng chờ	mọi trang qua trình duyệt, đảm bảo sign in	`7637ba2`	`9cc37c4`
API danh mục trả 403 cho request	`list -r` rỗng	đọc JSON trong `<pre>` qua trình duyệt, một `fetch_catalog` cho ba lớp	`61f2cbf`	`ae21659`
Loại activity mới: `html_bundle`, `badge`, `credential`	bị bỏ qua trong im lặng	chuyển hướng bằng `dict`, loại lạ giữ nguyên và log	`31390c6`	`9cfcd03`
`ld+json.hasPart` gắn nhãn Lab thành Course	Path trỏ sai Course	`ql-contents-menu` mới là nguồn sự thật, `ld+json` chỉ bổ sung	`0b3c374`	`1167943`
Tên Course có dấu nháy đơn	frontmatter YAML hỏng, Obsidian báo lỗi	`json.dumps` cho giá trị vô hướng, `title` thay `name`	`1aa2f3e`	`23d5a63`

Bảng 9: Khi Nền Tảng Thay Đổi Và Hành Vi

Mỗi dòng của bảng là một lần con người quyết định "thay đổi này xứng đáng để được xử lý"; không có dòng nào do công cụ tự sửa (phán đoán và hành động). Những gì chưa được triển khai (ví dụ duyệt partner portal, tự động phần Lab) cũng là quyết định có lý do và được trình bày ở Phần 5.2.

4.4. Cấu Hình Và Tính Di Động

NFR-03 và NFR-06 được đáp ứng bằng `config.py` với ba lớp cấu hình, lớp sau ghi đè lớp trước:

1. Mặc định trong mã (`DEFAULTS`):
   - `data/`, `csbmdvault/`, `.webdriver_profiles/`, Chrome, không headless.
   - Đủ để chạy ngay sau `uv sync` một cách mặc định.
2. `config.yaml` (dữ liệu, không phải mã):
   - Tìm theo `$CSB_CONFIG`, rồi `./config.yaml`, rồi `./config/config.yaml`.
   - Đường dẫn tương đối tính từ thư mục chứa file, nên `vault: ../skills-vault` nghĩa là một chỗ bất kể chạy lệnh từ đâu.
   - Repo kèm `config.example.yaml` có chú thích từng khoá.
3. Biến môi trường `CSB_DATA`, `CSB_VAULT`, `CSB_PROFILE_DIR`, `CSB_BROWSER`, `CSB_HEADLESS`:
   - Ghi đè từng khoá, tiện cho script và CI.
   - Chuỗi `"1"`, `"true"`, `"yes"`, `"on"` được hiểu là bật.

```
    settings = {section: dict(values) for section, values in DEFAULTS.items()}
    base_dir = Path.cwd()

    config_file = find_config_file()
    if config_file:
        base_dir = config_file.parent
        with open(config_file, encoding="utf-8") as yamlfile:
            user = yaml.safe_load(yamlfile) or {}
        for section, values in user.items():
            if section in settings and isinstance(values, dict):
                settings[section].update(values)

    for (section, key), env_name in ENV_KEYS.items():
        if env_name in os.environ:
            settings[section][key] = os.environ[env_name]

    for key, value in settings["paths"].items():
        settings["paths"][key] = (base_dir / str(value)).resolve()

    headless = settings["browser"]["headless"]
    if isinstance(headless, str):
        settings["browser"]["headless"] = headless.strip().lower() in ("1", "true",
"yes", "on")

    return settings
```

Mã nguồn 40: `config.load_settings()` – Tìm và Tải `config.yaml`

Cho tới v2.0.0, cấu hình được lưu trong `settings.py`, vốn là mã Python chứa đường dẫn tuyệt đối của máy cá nhân. Các thay đổi trên giúp repo chạy được trên máy khác ngay sau khi đồng bộ về mà không cần sửa mã nguồn. Đây là phần "mở" và "khả chuyển".

4.5. Kết Chương

Chương này trình bày các nội dung mở rộng có thể không hoàn toàn nằm trong bộ khung kiến thức của môn học IE221, nhưng là các phần phần không thể thiếu hoặc/và đã có từ những ngày đầu của công cụ, vì vậy sẽ được thêm vào vào để bổ sung và xây dựng nên tính đầy đủ của báo cáo.

CHƯƠNG 5.

KẾT LUẬN

Trong Chương cuối này, sinhv viên thực hiện xin đối chiếu kết quả với hai bộ yêu cầu đã đặt ở Chương 1 so với các Chương của môn học IE221, từ đó trình bày những hạn chế còn lại tính đến ngày hôm nay, và nêu hướng phát triển về sau. Đồng thời cũng nêu lên những kinh nghiệm đúc kết được từ quá trình phát triển công cụ như một dự án cá nhân và trong góc nhìn của Thiết Kế Hệ Thống (System Design).

5.1. Kết Quả Đạt Được

5.1.1. Đối Chiếu Với Yêu Cầu

Trạng thái ghi theo ba mức: **Đạt**, **Một phần**, **Chưa**.

Mã	Trạng thái	Bằng chứng
FR-01	Đạt	`list -p` đọc offline; `list -p -r` làm mới qua trình duyệt (macOS); `fetch_catalog` (Mã nguồn 25)
FR-02	Đạt	`fetch -p 280 --cascade --toc` (macOS)
FR-03	Đạt	`fetch -c 892 --toc` (macOS); bốn bước của `extract_transcript` ở Mục 3.6
FR-04	Một phần	`fetch -l ID` được nhận nhưng in "not yet fully supported": lab chỉ được thu thập qua course chứa nó, có nghĩa là mối quan hệ trực tiếp trong cây Tổ Chức Dữ Liệu (`process_lab` , Mã nguồn 17); `md -l` và `list -l` chạy được
FR-05	Đạt	`extract_course_metadata` so sánh `datePublished` (Mã nguồn 31)
FR-06	Đạt	`md -c 892 --toc` sinh note ở Mã nguồn 34; `generate_markdown` ghi đè ở Path, Course, Lab
FR-07	Đạt	`search vertex` tìm thấy path 280 và course 892
FR-08	Đạt	`reindex` dựng lại `index.json`
FR-09	Đạt	`process_document` tải file pdf vào `materials/courses/<id>/` với cookie của trình duyệt
FR-10	Một phần	`Course.generate_prompt` sinh `<id>-prompt.json` , gọi được từ TUI (`g`), chưa có lệnh CLI; tạm ngừng theo ghi chú ở Bảng 3
FR-11	Đạt	`skills-scraper-tui` mở menu màu trong Terminal và thoát không lỗi (Hình ảnh 6)
NFR-01	Đạt	ba cơ chế ở Phần 4.2

Mã	Trạng thái	Bằng chứng
NFR-02	Đạt	JSON UTF-8 có thụt lề, Markdown thuần; `git diff` là cách phát hiện thay đổi
NFR-03	Đạt	ba lớp cấu hình ở Mã nguồn 40, `config.example.yaml` có chú thích
NFR-04	Đạt	`try/except` theo từng activity; loại lạ giữ nguyên và báo (Mã nguồn 22); bỏ qua course có nêu rõ lý do
NFR-05	Đạt	`write_text` nguyên tử (Mã nguồn 26)
NFR-06	Đạt	`uv sync` rồi test phải xanh từ clone sạch trên hệ điều hành khác/máy khác (Mã nguồn 39)

Bảng 10: Kết Quả Đạt Được

Hai mục "Một phần" đều có cùng một nguyên nhân: ở v2.0.0 chúng cũng chưa hoàn chỉnh (chưa đủ độ cấp thiết), và trong phạm vi đồ án sinh viên ưu tiên làm cho những gì đã có chạy đúng với site hiện tại hơn là mở thêm tính năng. Cả hai được giữ nguyên trong mã nguồn với thông báo tường minh thay vì bị giấu đi trong im lặng.

5.1.2. Đối Chiếu Với Chương Bài Giảng

Tương tự, các kiến thức của môn học được phản ánh trong mã nguồn như các ví dụ dưới đây. Đồng thời, phần nào được bỏ qua sẽ có ghi chú *tường minh* tương ứng.

Chương	Nội dung	Bằng chứng trong mã
01 *[8]*	Môi trường Python	`pyproject.toml`, `.python-version`, `uv.lock` (Mã nguồn 3)
02 *[9]*	Kiểu dữ liệu	`dict` lồng modules/steps/activities; f-string; `tuple` làm khoá sắp xếp
03 *[10]*	Rẽ nhánh	chuyển hướng bằng `dict` theo loại activity; bỏ qua theo `datePublished` (Mã nguồn 21)
04 *[11]*	Vòng lặp	phân trang `while` có điểm ngắt `MAX_PAGES` (Mã nguồn 25)
05 *[12]*	Hàm	tham số mặc định, `**kwargs`, đệ quy `_parse_lesson_item` (Mã nguồn 23)
06 *[13]*	Module, package	`src/skills_scraper/` với ba sub-package, `__init__.py` Mã nguồn 7
07 *[14]*	Ngoại lệ	bắt ngoại lệ theo activity, `try/except/raise` khi ghi nguyên tử, `KeyboardInterrupt` (Mã nguồn 26)
08 *[15]*	Hướng đối tượng	`Serialize`, `BaseEntity`, `Collection`, `@property`, `super()`, ghi đè, composition (Hình ảnh 3)
09 *[16]*	Giao diện đồ hoạ	không dùng, có lý do ở Phần 2.6; TUI thay thế
10 *[17]*	Cơ sở dữ liệu	không dùng, có lý do ở Phần 2.6; file JSON và `index.json` dựng lại được

Bảng 11: Đối Chiếu Với Kiến Thức Môn Học

5.2. Hạn Chế

Công cụ còn những giới hạn sau, tất cả đều được nói ra trong mã nguồn hoặc tài liệu kèm repo, hoặc bản thân báo cáo này:

- Cần một phiên đăng nhập thật trên trình duyệt, nên không chạy hoàn toàn không người dùng trực tiếp; `--headless` chỉ dùng được sau khi profile đã đăng nhập và được dùng lại.
- Chưa thu thập lab độc lập (FR-04) và chưa có lệnh CLI cho tệp prompt (FR-10).
- Chưa thử lại khi mất kết nối vì cơ chế *tất cả hoặc không gì cả*: một lần rớt kết nối giữa `--cascade` dừng course đang chạy, chạy lại lệnh sẽ tiếp từ course đó nhờ idempotent.
- Bảo trì: Vẫn sử dụng `print()` trong `model/` thay cho `logging`.
- Chất lượng mã nguồn: `ruff check` (theo PEP 8 [18]) còn nhiều cảnh báo, đa số về kiểu (`implicit-optional`, `blind-except`, thứ tự import), chưa có lỗi logic.
- Tên file Markdown thực tế lấy từ tiêu đề của thực thể nên khi Google đổi tiêu đề thì liên kết trong vault bị gãy (orphaned/broken); checkbox tiến độ người học tự đánh bị ghi đè khi sinh lại Markdown (cần kiểm tra sự thay đổi khi cần).
- Partner portal không còn kiểm thử được vì sinh viên không còn quyền truy cập.

5.3. Hướng Phát Triển

Theo thứ tự nên làm, mỗi mục là một hoặc vài commit nhỏ trên cùng kiến trúc, không mục nào cần phải sửa `model/` về căn bản, trừ khi Google có một sự thay lớn về Cấu Trúc Dữ Liệu của nội dung trên website.

1. Thử lại khi mất kết nối giữa chừng (P10) và tái dùng cửa sổ Chrome đang mở.
2. Bundle OKF đầy đủ: lệnh `index` sinh `index.md` cho mỗi thư mục và `log.md` ở gốc vault.
3. Tên file dạng `<id>-<slug>.md` để liên kết không gãy khi đổi tiêu đề, có nghĩa sự nhất quán đến mức các file Markdown xuất ra.
4. Đổi `print()` sang `logging` để tăng khả năng bảo trì.
5. Xử lý các cảnh báo từ `ruff check` để tăng chất lượng mã nguồn.
6. Thu thập lab độc lập và lệnh `prompt` trên CLI, hoàn thành khép lại hai mục "Một phần".
7. Portal khác (AWS Skill Builder, vv..) bằng cấu hình và lớp con ghi đè phần phân tích trang, như cách `Paths` ghi đè `API_URL`.
8. Dùng vault làm kho tri thức cho AI agent qua plugin Obsidian hoặc MCP, với tệp prompt JSON làm đầu vào tóm tắt; đây là nơi khung CODE đi hết bước **Distill**.

Tuy nhiên, sinh viên kỳ vọng một sự đóng góp từ bên ngoài hơn là tự thực hiện, vì cơ bản công cụ hiện đang phục vụ được nhu cầu cá nhân nên thiếu động lực để phát triển tiếp.

5.4. Các Bài Học Từ Dự Án

- Trên một hệ thống lớn, bản chất là sự hợp tác (collaboration) của các công cụ nhỏ qua các dịch vụ điều phối (orchestrator).
- Chia nhỏ vấn đề đến mức có thể xử lý một cách rành mạch mà không cần giải thích quá nhiều.
- "Đừng lặp lại", tái sử dụng các thành phần đã có.
- Giữ sự đơn giản, để có thể luôn luôn thay thế khi cần.
- Tạo sự hiệu quả, để không phải đầu tư (thời gian, công sức) vào việc thay thế.
- Làm gì, hoặc không làm gì, đều cần được miêu tả rõ ràng.

5.5. Lời Kết

Đồ án xuất phát từ một nhu cầu thật của sinh viên trong công việc và được giải bằng một công cụ nhỏ, cố ý nhỏ: thu thập nội dung Google Skills về file JSON và sinh một vault Markdown cho Obsidian, với hai runner trên cùng một lõi. Về mặt học thuật, mỗi chương của môn *IE221 – Kỹ Thuật Lập Trình Python* đều được thể hiện trong mã nguồn (Bảng 11), với những khối kiến thức không được sử dụng, báo cáo cũng đã trình bày các quyết định không dùng với lý do rõ ràng; về mặt kỹ thuật, 15 trong 17 yêu cầu đạt, hai yêu cầu còn lại đạt một phần và được nói thẳng (Bảng 10).

Điều sinh viên mang theo từ đồ án không phải là bản thân scraper, vốn sẽ phải đổi theo site, mà là ba thói quen: *dữ liệu để lại phải sống lâu hơn công cụ*, *lỗi ồn ào tốt hơn lỗi im lặng*, và *mỗi quyết định dù là "không làm" cũng phải có lý do ghi lại được.* Đó cũng là cách sinh viên học được và đang xây các thành phần nhỏ (micro), không trạng thái (stateless) cho những hệ thống lớn hơn trong công việc.

PHỤ LỤC A

Công Cụ Sử Dụng

A.1 Mã Nguồn

- Python 3.12
- uv
- Selenium 4
- Beautiful Soup 4
- requests
- PyYAML
- pytest
- ruff (language server trong ZED Editor)
- Chrome
- Obsidian.

A.2 Báo Cáo

- Typst 0.15.1: Một typesetting mã nguồn mở, thay thế cho LaTex.
  - Báo cáo và slides được với theme `uit-theme` của sinh viên.
  - Bản dòng chính (master): sam-uit/typst-report-template (đã thay đổi rất nhiều với phiên bản đang sử dụng trong báo cáo này).
  - Vốn được sử dụng tại các môn học khác như sam-uit/ie103-project-report/tree/main/baocao

A.3 Khai Báo Sử Dụng AI

- Claude:
  - Hỗ trợ rà soát mã nguồn và soạn tài liệu
  - Thực hiện một số thay đổi trên nhánh `refactor-ie221`
  - Không thay đổi phần lõi `/model` đặc biệt các phần về cấu trúc và phân cấp dữ liệu.

PHỤ LỤC B

Nguồn Gốc Và Tư Liệu

B.1 Mã Nguồn

- Kho mã nguồn (repo): github.com/ggcta/skills-scrapper, nhánh `refactor-ie221`.

Ứng dụng trong báo cáo là branch `refactor-ie221` của repo, tách từ tag `v2.0.0` (commit `ac49ba2`, 18/02/2026), công cụ hoàn toàn là công trình/dự án của sinh viên, bắt đầu từ 05/01/2025 dưới tên Cloud Skills Boost helper. Dòng chính (master) của repo đã chuyển sang lõi Go và giao diện Tauri (tới v2.17.0); báo cáo này chỉ xét phần Python và chỉ dùng dòng chính làm nguồn tham khảo cho các thay đổi của site.

> **Ghi Chú**
>
> - `ggcta` (Google Cloud Technical Architect), là một tổ chức trên Github, được tạo bởi sinh viên, phục vụ cho công việc, repo ở đây nhưng vẫn dưới quyền sở hữu hoàn toàn của sinh viên.
> - Tên người đóng góp có thể là `samdx`, `samdinhgg`, hoặc `sam-uit`, đều là các username khác của sinh viên ở các vai trò khác nhau.

B.2 Danh Sách Commit Refactor

Đây là các commit để chuẩn hóa theo nội dung kiến thức của môn học IE221 nhưng không thay đổi về phần lõi của mã nguồn, ví dụ: database/GUI đã không sử dụng thì sẽ vẫn không thêm vào khi CLI/TUI là quá đủ cho nhu cầu ban đầu và hiện tại của công cụ. Bao gồm:

- Chuyển sang `uv` và bỏ hẳn `requirements.txt`.
- Đổi thư mục chính từ `app/` sang `src/skills_scraper` theo chuẩn đóng gói của một gói Python.
- Bỏ Web UI (vốn là một phiên bản khác của GUI) và chỉ tập trung vào CLI/TUI.
- Và một số thay đổi khác.

Theo dõi bằng cách:

```
1 git log --oneline v2.0.0..refactor-ie221
```

```
...
f864415 drop the Flask web UI and the dead code around it
84c5d98 move app/ into src/skills_scraper, real package now
879de08 uv: pyproject + lock, bye requirements.txt
```

Mã nguồn 41: `git log --oneline v2.0.0..refactor-ie221` – Ví Dụ Các Thay Đổi

B.3 Kịch Bản Demo

Sinh viên khuyến nghị một kịch bản demo gồm năm bước như dưới đây, 5 tới 7 phút, theo thứ tự offline trước, mạng sau, và nếu kết nối mạng hay đăng nhập có vấn đề thì phần cốt lõi vẫn được thể hiện.

Bước	Lệnh	Điều cần chỉ ra
1	`uv run skills-scraper list -p`	đọc offline từ `data/`, không mở trình duyệt
2	`uv run skills-scraper md -c 892 --toc`	note sinh ra trong vault, frontmatter OKF, mở trong Obsidian thấy backlink tới path
3	`uv run skills-scraper fetch -c 892 --toc`	phiên đăng nhập một lần, log từng activity; chạy lại lần hai thì bỏ qua và in lý do
4	`uv run skills-scraper-tui`	cùng lõi, giao diện khác; thoát bằng `q`, Chrome đóng theo
5	vòng `for` shell, `jq`, `git diff --stat` (Mã nguồn 36)	ghép được với công cụ khác mà không cần API

Bảng 12: Phụ Lục – Kịch Bản Demo

B.4 Chạy Từ Bản Nộp Bài

Giải nén, rồi trong thư mục gốc của repo:

```
# Python 3.12, thư viện, hai lệnh skills-scraper*
uv sync

# 15 test offline, không cần kết nối mạng
uv run pytest

# help
uv run skills-scraper --help

# sinh một note từ dữ liệu mẫu trong repo
CSB_DATA=docs/samples/data CSB_VAULT=/tmp/vault \
  uv run skills-scraper md -c 892 --toc

# tuỳ chọn: nơi đặt data/ và vault
cp config.example.yaml config.yaml

# mở Chrome, đăng nhập một lần
uv run skills-scraper list -p -r
```

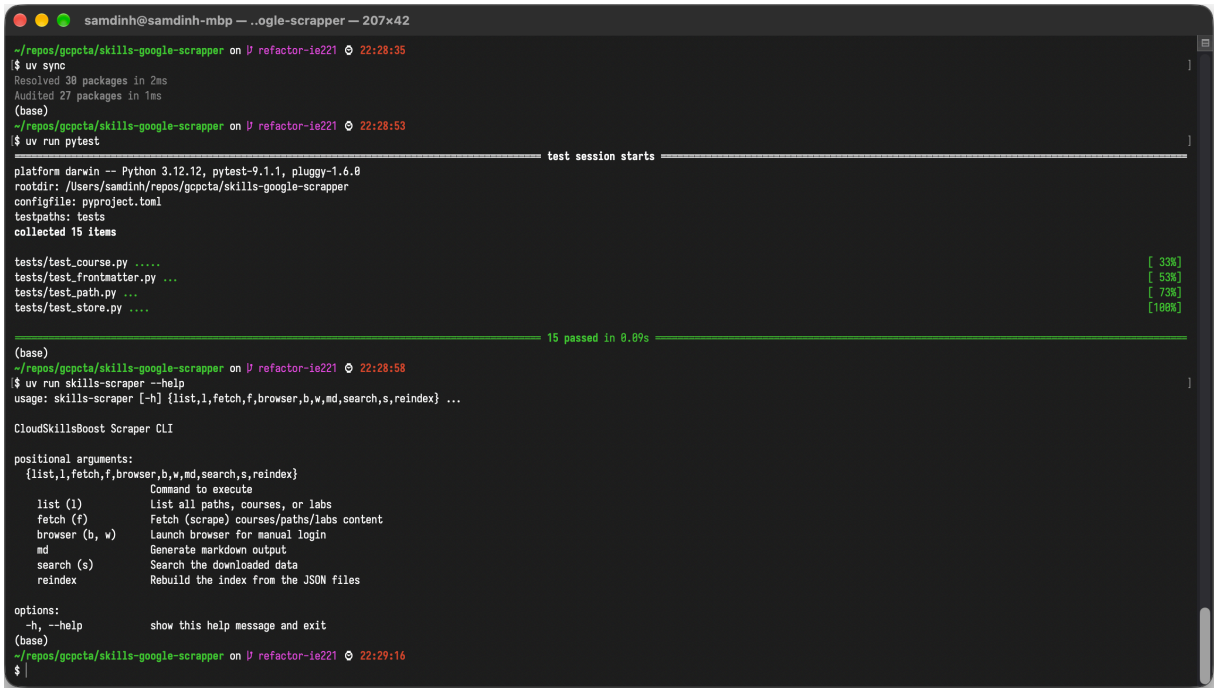
Mã nguồn 42: Phụ Lục – Các Lệnh Để Chạy Từ Bản Nộp Kèm

PHỤ LỤC C

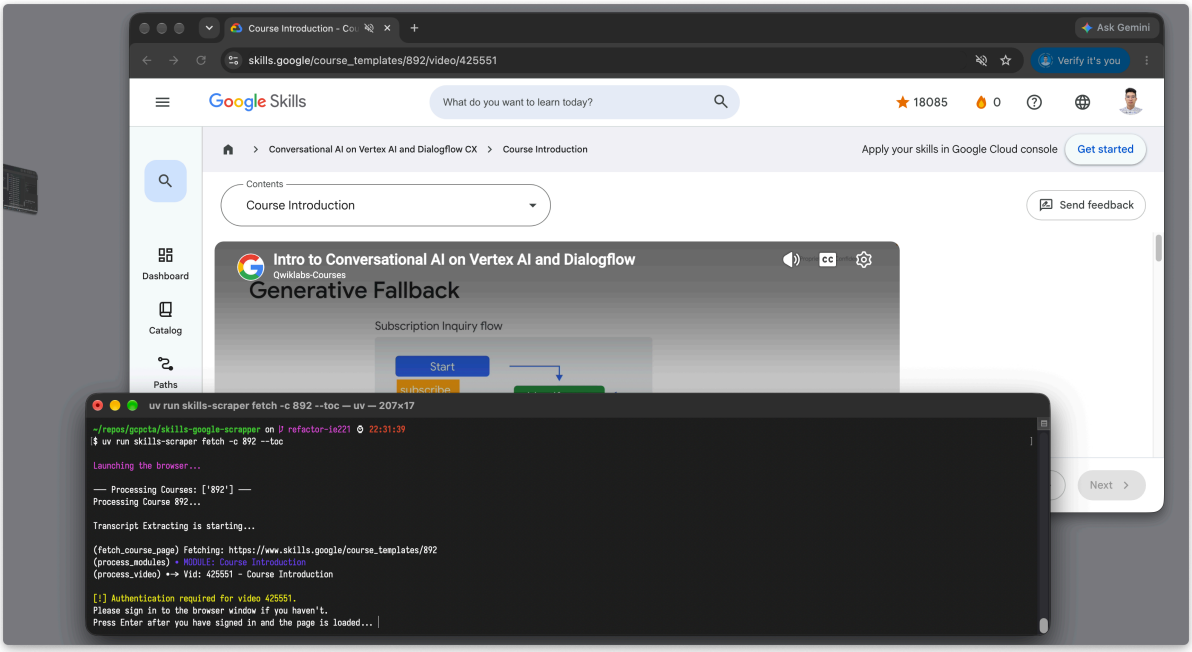
DEMO

Phần DEMO được tách thành Phụ Lục để tránh loãng nội dung chính và sẽ được trình bày theo một số ví dụ như dưới đây ở bản CLI. Bản TUI cơ bản là tương tự, chỉ khác là sử dụng menu chọn từ Terminal.

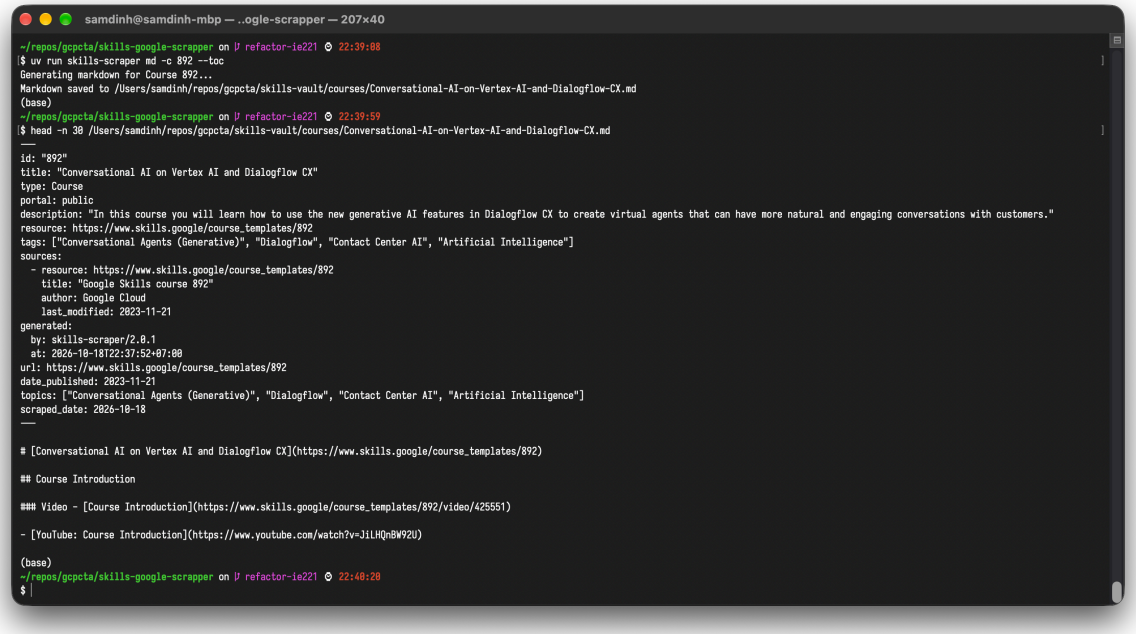
- `uv sync` và test, sau đó chạy thử bản CLI:



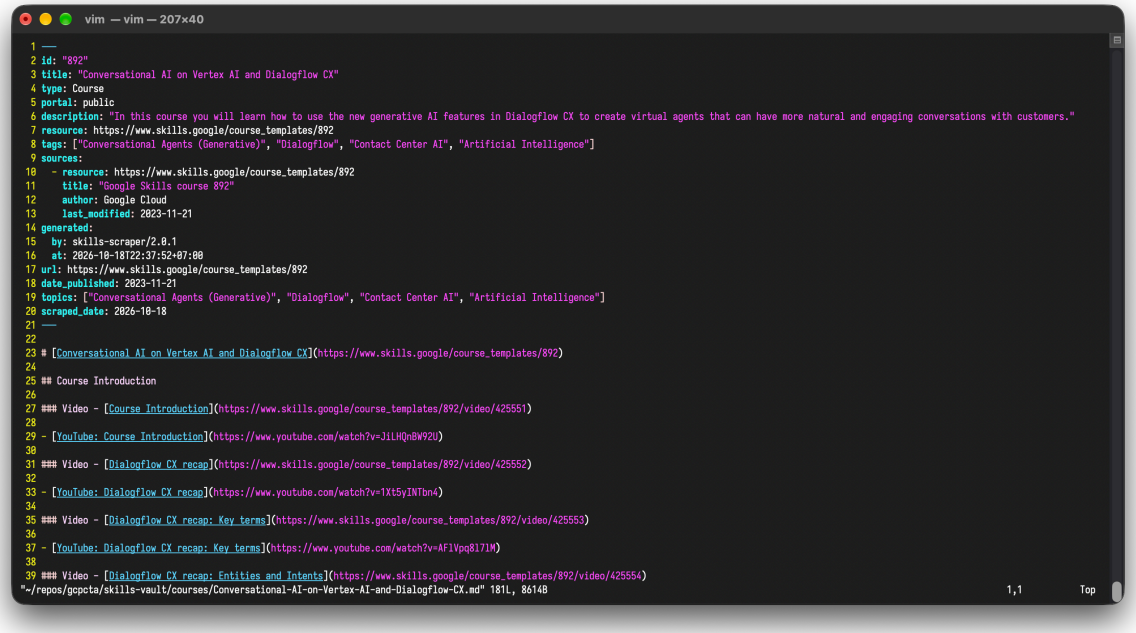
- `fetch -c` tải một khóa học cụ thể: chạy trình duyệt, yêu cầu đăng nhập.



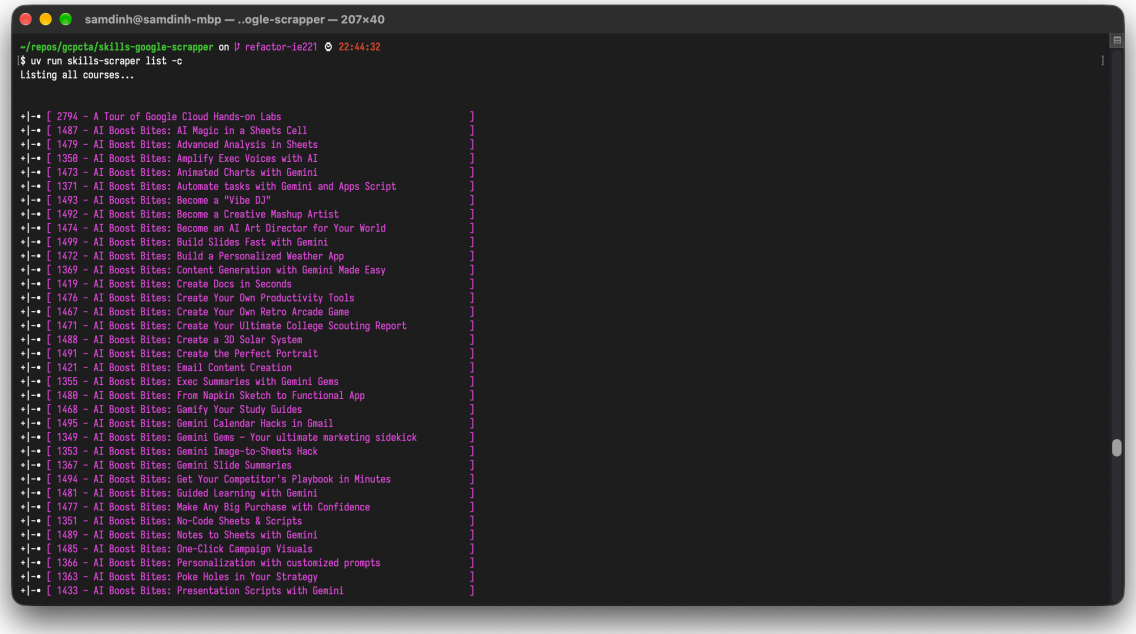
- `md -c 892 --toc` tạo file .md chỉ chứa bộ khung nội dung (Table Of Content)



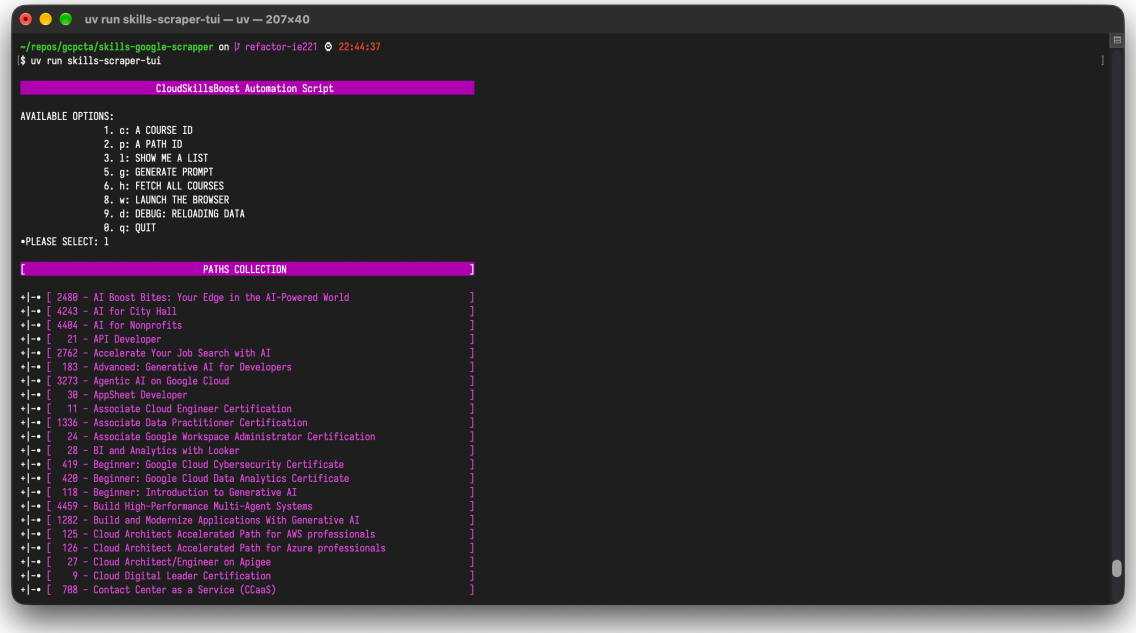
- Kết quả của `md -c 892 --toc`



- `list -c` liệt kê danh sách các Course đã biết, hoàn toàn có thể kết hợp với các *pager* (chia trang nội dung) như `less`, `more`, vv.. cho các nhu cầu cụ thể.



- TUI: liệt kê sử dụng menu tương tác, nên khó kết hợp với các *pager*.


uv run skills-scraper-tui — uv — 207×40
~/repos/gcpcta/skills-google-scrapper on refactor-ie221 22:44:37
$ uv run skills-scraper-tui
CloudSkillsBoost Automation Script
AVAILABLE OPTIONS:
1. c: A COURSE ID
2. p: A PATH ID
3. l: SHOW ME A LIST
5. g: GENERATE PROMPT
6. h: FETCH ALL COURSES
8. w: LAUNCH THE BROWSER
9. d: DEBUG: RELOADING DATA
0. q: QUIT
•PLEASE SELECT: 1
PATHS COLLECTION
[2480 - AI Boost Bites: Your Edge in the AI-Powered World
[4243 - AI for City Hall
[4404 - AI for Nonprofits
[21 - API Developer
[2762 - Accelerate Your Job Search with AI
[183 - Advanced: Generative AI for Developers
[3273 - Agentic AI on Google Cloud
[30 - AppSheet Developer
[11 - Associate Cloud Engineer Certification
[1336 - Associate Data Practitioner Certification
[24 - Associate Google Workspace Administrator Certification
[28 - BI and Analytics with Looker
[419 - Beginner: Google Cloud Cybersecurity Certificate
[420 - Beginner: Google Cloud Data Analytics Certificate
[118 - Beginner: Introduction to Generative AI
[4459 - Build High-Performance Multi-Agent Systems
[1282 - Build and Modernize Applications With Generative AI
[125 - Cloud Architect Accelerated Path for AWS professionals
[126 - Cloud Architect Accelerated Path for Azure professionals
[27 - Cloud Architect/Engineer on Apigee
[9 - Cloud Digital Leader Certification
[708 - Contact Center as a Service (CCaaS)

Tài Liệu Tham Khảo

[1] E. S. Raymond, *The Art of UNIX Programming*. Addison-Wesley, 2003.

[2] T. Forte, *Building a Second Brain*. Atria Books, 2022.

[3] Google Cloud Platform, "Open Knowledge Format Specification v0.2". [Online]. Có tại: https://github.com/GoogleCloudPlatform/open-knowledge-format

[4] Astral, "uv Documentation". [Online]. Có tại: https://docs.astral.sh/uv/

[5] Python Packaging Authority, "Python Packaging User Guide". [Online]. Có tại: https://packaging.python.org/

[6] Selenium Project, "Selenium Documentation". [Online]. Có tại: https://www.selenium.dev/documentation/

[7] L. Richardson, "Beautiful Soup Documentation". [Online]. Có tại: https://www.crummy.com/software/BeautifulSoup/bs4/doc/

[8] Nghi Hoàng Khoa, "IE221 Kỹ Thuật Lập Trình Python, Chương 01: Giới thiệu Python và môi trường". Trường Đại Học Công Nghệ Thông Tin, ĐHQG-HCM, 2026.

[9] Nghi Hoàng Khoa, "IE221 Kỹ Thuật Lập Trình Python, Chương 02: Kiểu dữ liệu". Trường Đại Học Công Nghệ Thông Tin, ĐHQG-HCM, 2026.

[10] Nghi Hoàng Khoa, "IE221 Kỹ Thuật Lập Trình Python, Chương 03: Cấu trúc rẽ nhánh". Trường Đại Học Công Nghệ Thông Tin, ĐHQG-HCM, 2026.

[11] Nghi Hoàng Khoa, "IE221 Kỹ Thuật Lập Trình Python, Chương 04: Vòng lặp". Trường Đại Học Công Nghệ Thông Tin, ĐHQG-HCM, 2026.

[12] Nghi Hoàng Khoa, "IE221 Kỹ Thuật Lập Trình Python, Chương 05: Hàm". Trường Đại Học Công Nghệ Thông Tin, ĐHQG-HCM, 2026.

[13] Nghi Hoàng Khoa, "IE221 Kỹ Thuật Lập Trình Python, Chương 06: Module và package". Trường Đại Học Công Nghệ Thông Tin, ĐHQG-HCM, 2026.

[14] Nghi Hoàng Khoa, "IE221 Kỹ Thuật Lập Trình Python, Chương 07: Xử lý ngoại lệ". Trường Đại Học Công Nghệ Thông Tin, ĐHQG-HCM, 2026.

[15] Nghi Hoàng Khoa, "IE221 Kỹ Thuật Lập Trình Python, Chương 08: Lập trình hướng đối tượng". Trường Đại Học Công Nghệ Thông Tin, ĐHQG-HCM, 2026.

[16] Nghi Hoàng Khoa, "IE221 Kỹ Thuật Lập Trình Python, Chương 09: Giao diện đồ hoạ (tkinter, PyQt)". Trường Đại Học Công Nghệ Thông Tin, ĐHQG-HCM, 2026.

[17] Nghi Hoàng Khoa, "IE221 Kỹ Thuật Lập Trình Python, Chương 10: Cơ sở dữ liệu (MySQL, MS SQL)". Trường Đại Học Công Nghệ Thông Tin, ĐHQG-HCM, 2026.

[18] G. van Rossum, B. Warsaw, và A. Coghlan, "PEP 8: Style Guide for Python Code". [Online]. Có tại: https://peps.python.org/pep-0008/